

Replacing Large Language Models with Jev Decision Models for Low-Latency Edge Service Orchestration

Delong Li, Xu Wang, Haochen Gong, Rui Lang, and Guangsheng Yu
*School of Electrical, Mechanical and Biomedical Engineering
 University of Technology Sydney, Sydney, Australia*

OniReimu/Edge-Computing-JEV

OniReimu/Edge-Computing-JEV

Abstract—Natural-language service requests can require a language-model decision before execution starts, consuming part of the request’s latency budget. We integrate Jev’s decision-oriented application programming interface (API) into edge service orchestration to reduce this overhead while retaining service completion. The integration extracts four to eight bounded intent fields and applies a shared validator, admission policy, and scheduler, accounting for decision waiting throughout the request timeline. We compare Jev, two self-hosted decision models, and three hosted large language models (LLMs) on 8,280 verified requests and on a live admission path with modeled execution and a real optical character recognition service. Across 33 test conditions, Jev reduces median decision latency by 22.7–64.5% relative to the fastest LLM. This latency barely moves with input size, contract width, or catalog size. On four-field contracts, Jev’s API fees per correct decision are 59.7–80.9% lower at a cost of a few exact-match points, while wide contracts mark the limit of the substitution. Receiving the service catalog with each request, Jev names unseen services as accurately as known ones. On the live admission path, Jev keeps 0.91–0.95 of requests exact and on time at loads where the LLMs fall below 0.1. Since caching repeated descriptions gives the interpreters nearly the same latency, Jev’s gain lies in fresh decisions. These results support decision-model substitution for latency-bound admission on bounded contracts.

Index Terms—Large language models, decision models, service orchestration, edge computing, service admission, quality of service.

1. Introduction

Natural-language interfaces let users describe an edge service and its execution requirements together. One such request asks the service to read the text in an image, keep the image at its originating site, and return the result before a deadline. Turning that request into an executable job requires both semantic interpretation and a placement decision. In edge computing, communication, resource availability, and placement jointly determine whether the service responds in time [1], [2], [3]. An interpreter on the admission path con-

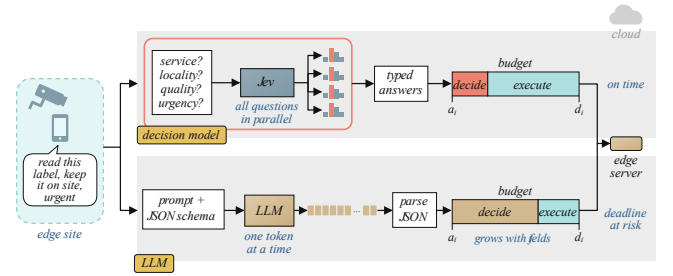


Figure 1. Two ways to interpret the same edge request. A camera or phone at an edge site asks for a service that must run on the site’s edge server. A decision model such as Jev-1.13.0 scores every contract question in parallel against the request text and returns a typed answer with a probability distribution over the declared options. Because an LLM decodes a JSON object one token at a time, its decision time grows with the number of fields. The bars sketch how much of the budget between arrival a_i and deadline d_i each interpreter leaves for execution.

sumes part of that same response budget, delaying execution and leaving less time to complete the service.

For a bounded service catalog, interpretation often requires only a small set of decisions. These decisions include service type, placement permission, quality tier, and urgency. A large language model (LLM) can express these decisions through a short structured response, but the downstream scheduler needs the selected values rather than a generated explanation. A decision-oriented model could reduce this overhead even relative to a generative model configured for concise output (Fig. 1). At the service level, the goal is to reduce this decision overhead while preserving correct, on-time completion under the request’s execution requirements.

Intent-based networking already separates desired outcomes from their implementation [4]. Language-driven network systems and service prototypes connect interpretation to configuration and orchestration [5], [6], [7], [8], [9]. Small models and discriminative extraction offer further ways to specialize bounded language tasks [10], [11]. We examine Jev, a decision-oriented service accessed through an application programming interface (API) [12], as the interpreter in an edge-service admission path. We examine two self-hosted decision models, SemIf-Qwen3.5-4B [13] and Laya [14], in the same role. Queueing, communication, and

execution all contribute to response time. Their interaction with interpretation determines how much a faster decision benefits the service.

We investigate five linked questions. The first four isolate the interpreter and ask how decision latency, semantic correctness, and billed API cost change as the input grows (RQ1) and as its wording degrades (RQ2). The same three quantities are measured as the contract gains fields (RQ3) and as the service catalog grows and changes (RQ4). The fifth asks whether the substitution lowers full response time while retaining correct service completion, under what workload conditions faster interpretation is most useful, and how it interacts with caching for repeated requests (RQ5).

We address these questions in two layers. The interpretation layer uses EdgeIntent v1, a benchmark of 8,280 English requests whose labels are fixed first and whose text is kept only when a blind verifier recovers every label. On it, three decision models face three hosted LLMs (DeepSeek-V4.1-Flash, GLM-5.3-Flash, and Qwen3.8-Flash) in every condition. The service layer combines live interpretation with modeled execution, followed by a real three-node optical character recognition (OCR) service. The real service transfers images, executes recognition, and checks the returned text against reference annotations. A fixed rule parser, a generative model that shares SemIf-Qwen3.5-4B’s weights, a sentence-embedding reranker, and a trained service classifier extend the comparison beyond the six interpreters.

The contributions are as follows.

- We integrate decision models into an edge-service admission system through a typed intent contract of four to eight fields with a shared validator and scheduler. The system places interpretation, queueing, communication, and execution on one timeline and judges each interpreter by service deadlines and completion.
- Three decision models and three hosted LLMs interpret the same contracts in 33 test conditions. We measure their decision latency, API fees per correct decision, and accuracy. The conditions vary the input size, the bundling of requests, and the width of the contract.
- A catalog test gives each interpreter the service catalog with every request and checks whether it names services unseen during development as accurately as known ones, with a frozen service classifier as the reference. The same test finds the option limit of each decision model.
- One cache policy, applied to every interpreter, separates the latency of fresh interpretation from that of repeated requests. The comparison locates where fast decision models reduce service latency once caching is in place.

2. Related Work

Table 1 lists the evaluation dimensions that the closest related studies report, with columns chosen for an interpreter in an edge admission path. Seven of the fourteen network-intent and orchestration studies report interpretation latency as a result, and two report its tail. LLNet alone targets energy, and none of the fourteen varies the offered load

or evaluates reuse of model outputs. Across all twenty-two studies, GLiNER2 alone evaluates options unseen in training and InferLine alone sweeps the request rate. No study reports more than six of the eleven dimensions as measured results. This study measures all eleven for the same interpreters.

2.1. Intent Interpretation and Service Orchestration

Intent-based networking separates desired outcomes from the mechanisms used to realize them [4]. Lumi translates natural-language network requirements through a learned interface and an intermediate representation [5]. NetConfEval evaluates language models on network-configuration tasks spanning policy translation, API calls, and configuration generation [6]. Work on fifth-generation (5G) mobile-network intent extraction likewise studies the interpretation stage [15]. LLNet and fine-tuned small-model network configuration bring more specialized models into this setting [7], [16]. These studies establish natural-language interpretation as a systems component whose accuracy and execution overhead both matter.

Several systems extend interpretation into service orchestration. Chat-Driven Optimal Management of Virtual Network Services combines an interpreter with optimization [8]. From Prompt to Service evaluates intent routing and real conversational service paths, including a fine-tuned small router [9]. Intent Engine uses intent grounding and validation within a network orchestration architecture [19]. Agentic infrastructure orchestration, zero-touch network pipelines, and DMO-GPT address broader automation workflows [17], [21], [23]. JAUNT studies intent-aware network tool routing [18], while grounded and role-based sixth-generation (6G) systems examine additional forms of orchestration and agent organization [20], [22].

Building on this work on service routing, we compare three decision models with three generative interpreters while holding the intent contract and downstream policy fixed. We measure completion against the returned service output, accounting for decision waiting, case-sensitive OCR correctness, and caching under the same policy for each backend. The decision unit is a user-issued service job, whose admission path can include hundreds of milliseconds of interpretation.

2.2. Bounded Decisions and Small Models

Efficient prediction need not require free-form generation. Sentence-BERT and SetFit support lightweight semantic representations and classification [31], [32]. GLiNER and GLiNER2 provide discriminative extraction approaches, with GLiNER2 also addressing structured extraction and classification [11], [33]. The argument for small models in agentic systems further motivates assigning bounded tasks to specialized components [10]. For networking decisions, NetLLM adapts language-model representations to networking tasks through task-specific components [24].

TABLE 1. EVALUATION COVERAGE OF THE CLOSEST RELATED WORK. ✓ MEASURED AND REPORTED AS A RESULT. ● MENTIONED, ENFORCED BY DESIGN, OR REPORTED ONLY FOR THE WHOLE PIPELINE. ✗ NOT ADDRESSED. UNSEEN CATALOG: OPTIONS ABSENT FROM TRAINING AND PASSED AT REQUEST TIME. SAFETY VIOLATIONS: MEASURED PLACEMENT, LOCALITY, OR POLICY VIOLATIONS IN THE DECISIONS. DEADLINE COMPLETION: WHETHER THE DOWNSTREAM SERVICE COMPLETES CORRECTLY OR ON TIME. COV. COUNTS ✓ CELLS. † ASSESSED FROM THE ABSTRACT.

Work	Paradigm	NL→ typed	Decision latency	Tail latency	Monetary cost	Energy or power	Unseen catalog	Safety violations	Deadline completion	Offered load	System testbed	Output reuse	Cov. (✓)
Network intent interpretation													
Lumi [5]	NER + intent grammar	✓	●	✗	✗	✗	✗	✗	✗	✗	✓	✗	2
Manias et al. [15]	Prompted LLM	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	1
NetConfEval [6]	LLM benchmark	✓	●	✗	✓	✗	●	●	●	✗	✓	●	3
LLNet† [7]	Fine-tuned SLM	●	✗	✗	●	●	✗	✗	✗	✗	✗	✗	0
Lira et al. [16]	Fine-tuned SLM	●	✓	●	●	✗	✗	●	✗	✗	✗	●	1
Intent-driven service orchestration													
Brodimas et al. [17]	Agentic LLM	●	●	✗	✓	✗	✗	●	✓	✗	✓	✗	3
JAUNT [18]	Agentic LLM router	✓	●	✗	✗	✗	●	✗	✓	✗	●	✗	2
Miyaoka et al. [8]	Classifier/LLM + ILP	✓	✓	✗	●	✗	✗	●	●	●	●	✗	2
Intent Engine [19]	Grounded LLM	✓	✓	✓	●	✗	●	✓	●	✗	●	✗	4
Martins et al. [20]	Agentic LLM	✓	✓	✗	●	✗	●	✓	✗	✗	✗	✗	3
DMO-GPT† [21]	Agentic LLM	●	✗	✗	✗	✗	●	✗	✗	✗	✓	✗	1
Nisiotis et al. [9]	Fine-tuned SLM router	✓	✓	✓	●	✗	●	●	●	✗	✓	✗	4
Parra-Ullauri et al. [22]	Agentic LLM	●	✓	✗	●	✗	●	●	✓	✗	✓	✗	3
Seo et al. [23]	LLM + RL agent	✓	✓	✗	●	✗	✗	✓	●	✗	●	✗	3
Bounded decisions and structured output													
NetLLM [24]	Adapted LLM + task head	✗	✓	●	●	✗	✗	●	✓	●	✓	✗	3
GLiNER2 [11]	Schema-driven encoder	✓	✓	✗	●	✗	✓	✗	✗	✗	✗	✗	3
JSONSchemaBench [25]	Constrained decoding	●	✓	✗	✗	✗	●	●	✗	✗	✗	✗	1
Model routing, caching, and serving													
Clipper [26]	Serving system	✗	✓	✓	●	✗	✗	✗	✓	●	✓	✓	5
InferLine [27]	Pipeline serving	✗	✓	✓	✓	✗	✗	✗	✓	✓	✓	✗	6
GPTCache [28]	Semantic cache	✗	✓	✗	●	✗	✗	✗	✗	✗	✗	✓	2
FrugalGPT [29]	LLM cascade	✗	●	✗	✓	●	✗	✗	✗	✗	✗	●	1
RouteLLM [30]	LLM router	✗	●	✗	✓	✗	●	✗	✗	●	✗	✗	1
This work	Decision models vs. LLMs	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	11

Because a trained classifier fixes its label set at training time, a service added to the catalog needs labelled examples and a new training run. Decision models that receive the catalog with each request avoid that step. RQ4 measures the difference against a DistilBERT service classifier [34] and a MiniLM sentence-embedding reranker [31], [35].

Alongside the hosted comparison, we include a fixed rule parser and an open generative model, Qwen3.5-4B [36], which shares its weights with SemIf-Qwen3.5-4B. Rules provide a low-overhead interpretation policy, while Qwen provides a self-hosted generative alternative under the same service contract. The two self-hosted arms therefore differ only in how the answer is read out.

2.3. Serving, Structured Output, and Cost

Serving systems such as Clipper and InferLine address inference deployment under latency and resource constraints [26], [27]. PagedAttention and SGLang improve important aspects of language-model serving [37], [38]. Consequently, a measured self-hosted model latency is a property of its serving configuration as well as its weights.

Structured-output validity also needs to be distinguished from task correctness. JSONSchemaBench evaluates efficiency, constraint coverage, and quality in structured generation [25]. A valid JavaScript Object Notation (JSON) object can still request the wrong service, just as a correctly interpreted request can yield incorrect OCR text. We retain these separate endpoints. Cost-aware prediction and routing systems such as FrugalML, FrugalGPT, and RouteLLM

consider different ways to allocate model calls under quality and cost objectives [29], [30], [39]. We compare fixed interpretation backends under a shared admission policy.

Caching is another direct alternative to repeatedly paying inference latency. GPTCache studies reuse through a semantic cache [28]. Our cache reuses valid parsing results for identical normalized intent text. Applying the same policy to every backend tests how decision reuse changes the value of faster interpretation.

3. Service Admission Architecture

3.1. Request and Interpretation Contract

Each request i contains a natural-language description, an arrival time a_i , an absolute deadline d_i , an origin, and payload metadata. Times share the originating controller’s clock. Only the description is sent to the interpreter. The scheduler receives the remaining metadata through the same interface for every backend. Reference intent labels and expected service outputs are available exclusively to the evaluator. Table 2 defines the shared interpretation contract. Its four core fields describe the requested service and execution requirements. With a catalog of K services, the service field has $K + 1$ values, and the core contract hence admits $27(K + 1)$ tuples. Two further tiers add retention and energy ($F = 6$), then redundancy and latency class ($F = 8$). Each added field maps to a scheduler action. Missing requirements remain unspecified rather than being inferred from the evaluator’s labels.

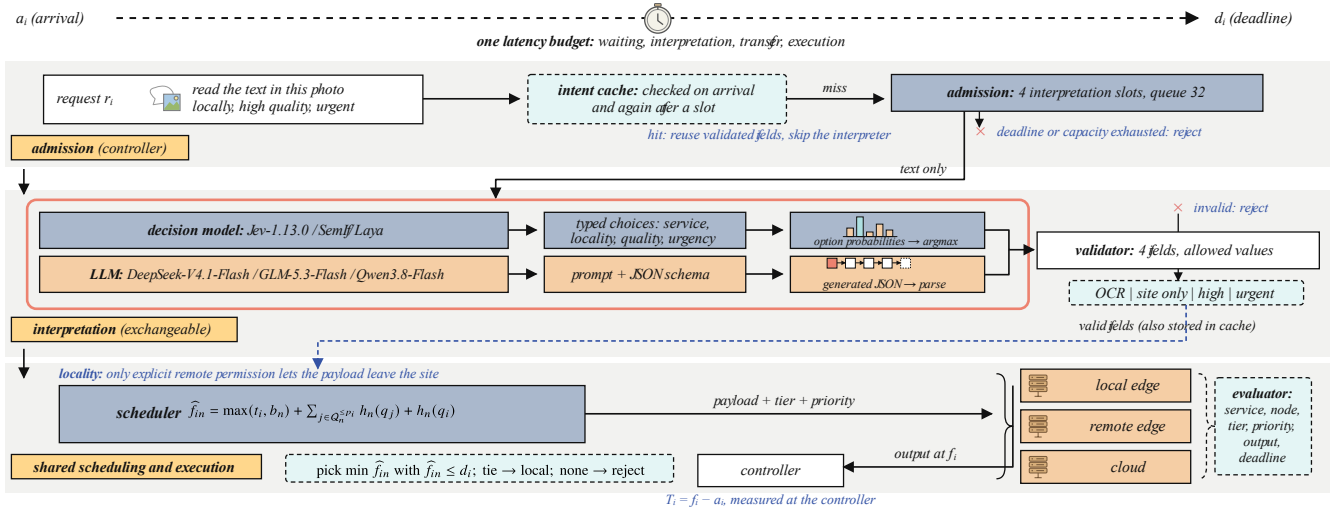


Figure 2. Admission path in detail. The controller checks the intent cache on arrival and again after an interpretation slot frees. The interpreter receives only the request text and returns typed choices or a JSON object, which the shared validator checks. The scheduler places each valid request with the predicted finish rule of Eq. (1). The evaluator compares service, node, tier, priority, output, and finish time with the reference requirements.

TABLE 2. SHARED INTENT CONTRACT. OCR DENOTES OPTICAL CHARACTER RECOGNITION. F IS THE NUMBER OF FIELDS IN THE CONTRACT.

Field	F	Allowed values
Service	4	Catalog of K services (count, detection, OCR, ...), unsupported
Locality	4	Site only, remote allowed, unspecified
Quality floor	4	Standard, high, unspecified
Urgency	4	Normal, urgent, unspecified
Retention	6	Discard after use, retain allowed, unspecified
Energy	6	Eco, performance, unspecified
Redundancy	8	Single, replicated, unspecified
Latency class	8	Real time, interactive, batch, unspecified

The decision models select among the allowed values, while the generative backends return the same fields in a concise structured response. The rule-based interpreter maps the same input text to this contract. The required meaning of each output is fixed across interfaces. For example, a request to read an image locally with high quality and urgent handling maps to OCR, site only, high, and urgent. Selecting the execution node remains the scheduler’s responsibility.

Each contract field becomes one choice question whose criteria text is shared verbatim with the LLM prompt and schema descriptions. Jev-1.13.0 answers all questions of a request in one hosted call. SemIf-Qwen3.5-4B reads the same questions out of Qwen3.5-4B weights, and Laya is a 421M-parameter typed-decision model. Both models run on the edge node. Because neither self-hosted model generates text, neither carries a per-token decoding cost. The catalog travels with the request as the options of the service question. A new service therefore reaches the interpreter without retraining, up to the number of options each model accepts.

3.2. Validation, Caching, and Admission

The validator checks that all required fields are present and that each value belongs to its allowed set. Invalid interpretations do not proceed to scheduling. Requests also leave the admission path when their deadline or the available admission capacity is exhausted. This common policy controls which responses can be executed. Semantic correctness must be evaluated separately against the request’s intended meaning.

When caching is enabled, each backend reuses its own validated interpretation for identical normalized request text under the same extraction policy. A response becomes reusable only after it has arrived. The cache contains neither reference labels nor service outputs, and it can retain a well-formed but semantically incorrect interpretation. Because repeated descriptions may accompany different images, reusing a decision does not eliminate the corresponding service execution. All backends use the same cache policy.

Admission and execution share one elapsed-time axis. Waiting for an interpretation slot and receiving a remote response both consume the budget between a_i and d_i . A call that outlives its request’s deadline continues to occupy its slot until the call terminates, while new requests continue to arrive. Interpretation latency thus affects both service feasibility and queue occupancy.

3.3. Shared Scheduling and Execution

After interpretation, the scheduler reads the current worker state. In the real service of RQ5 the workers are a local node, a second edge node, and a cloud node, each with its own link delay and bandwidth. Only an explicit remote permission allows a payload to leave its originating

site. Unspecified locality defaults to local execution. A high quality requirement selects the high tier, and other requests use the standard tier. Urgent requests take priority over ordinary requests without interrupting active work. These shared defaults can make distinct field values operationally equivalent, as with normal and unspecified urgency.

For the real service, let t_i denote the time at which request i is scheduled, after its interpretation has completed. Its selected tier is q_i , and its priority is p_i , with smaller values indicating higher priority. Let $\mathcal{N}_i$ contain the nodes that provide the interpreted service, satisfy its placement restriction, and have queue capacity. For a node $n \in \mathcal{N}_i$, let b_n be the estimated finish time of active work, or zero if idle. Let $h_n(q)$ be the calibrated duration of serving tier q , including transfer and response overhead. Finally, $\mathcal{Q}_n^{\leq p_i}$ is the set of pending jobs with equal or higher priority than request i , and q_j is pending job j 's tier. All node quantities refer to the state observed at t_i . The predicted finish time $\hat{f}_{in}$ is

$$\hat{f}_{in} = \max(t_i, b_n) + \sum_{j \in \mathcal{Q}_n^{\leq p_i}} h_n(q_j) + h_n(q_i). \quad (1)$$

The scheduler selects the node with the smallest predicted finish time among those satisfying $\hat{f}_{in} \leq d_i$, breaking ties in favor of the local node. It rejects the request if no such node exists. Equation 1 is a shared admission estimate. Execution-time variation and subsequent higher-priority arrivals can invalidate its prediction. Correct completion therefore depends on the observed result and finish time, not merely on passing the admission test.

An accepted request transfers its payload to the selected worker and returns the service output to the originating controller. Payloads are transferred only after node selection. The evaluator checks the actual service, node, tier, priority, output, and deadline against the reference requirements. All elapsed times are measured at the originating controller. Locality constraints govern service payloads, but the textual description still reaches the chosen interpretation API. The scheduler and these checks are common to all backends, allowing the comparison to focus on interpreter substitution.

4. Evaluation Methodology

4.1. Comparison and Evidence Design

The principal comparison is between decision models and LLMs configured for concise structured output without a generated reasoning trace. The decision models are Jev-1.13.0 (hosted), SemIf-Qwen3.5-4B, and Laya (self-hosted). The LLMs are DeepSeek-V4.1-Flash, GLM-5.3-Flash, and Qwen3.8-Flash, which are all hosted and use strict JSON schemas and temperature zero. All six appear in every condition and use the same intent contract, validator, admission limits, execution policy, and caching mechanism. References are evaluated alongside the six without entering their comparison. Qwen3.5-4B-JSON generates the JSON object from the same Qwen3.5-4B weights as

SemIf-Qwen3.5-4B, which isolates readout from generation at fixed weights and deployment. A fixed rule parser serves RQ1–RQ3. RQ4 adds a DistilBERT service classifier, frozen (DistilBERT-Clf-Frozen) or retrained on the changed catalog (DistilBERT-Clf-Retrained), and an all-MiniLM-L6-v2 sentence-embedding reranker (MiniLM-Reranker). Model identifiers, software, hardware, workload parameters, and execution procedures are given in Section 4.7.

RQ1–RQ4 form the interpretation layer. Because each sends identical request texts to every interpreter and scores the returned fields, only the interpreter changes between arms. RQ5 forms the service layer in two parts, as in our earlier exploratory study. Part A measures live decisions and combines their actual timing with modeled service execution. It evaluates the interpretation tradeoff and how decision waiting consumes execution slack. Part B executes a real three-node OCR service and checks the returned text. It tests whether lower decision latency remains useful after actual communication and processing. The measured hosted latency includes provider execution, routing, and network transport throughout.

4.2. The EdgeIntent v1 Benchmark

EdgeIntent v1 fixes each request's label tuple first, with balanced marginals per field, and has a generator model write the text second. A blind verifier then labels the text in a separate session without seeing the tuple, and a case is kept only when its labels match the tuple on every field. Lint checks reject texts that leak field names or enumeration values. The generators are Gemini-3.8-Flash and Claude-Opus-5.5 and the verifier is Claude-Opus-5.5. Both models lie outside the evaluated roster. The texts span six wording families, from operator tickets to voice-assistant utterances. Each of the 23 generated conditions has 300 test and 60 development cases, 8,280 in total. Ten further conditions are derived programmatically, for example by padding or noise. Development cases serve only prompt checks, classifier training, and threshold calibration. Table 3 lists the conditions of each RQ.

RQ1 pads each request with unrelated logs, ticket threads, or configuration dumps up to 16,384 tokens (RQ1a), and bundles $k \in \{1, 2, 4, 8\}$ requests into one message (RQ1b). RQ2 keeps the label tuples of the clean condition and rewrites their text as colloquial, noisy, negation-heavy, code-switched, default-baiting, self-revising, or key–value input. RQ3 crosses contract size $F \in \{4, 6, 8\}$ with low, medium, and high constraint density. RQ4 varies the catalog size $K \in \{4, 15, 64, 128, 254\}$ over nested catalogs of 254 edge services with deliberate near neighbors. Its churn conditions replace 25% or 50% of a 64-service catalog with services outside it, which the classifier never saw. Unsupported requests make up 10% of each RQ4 condition.

4.3. Service Workloads and Matched Comparisons

Part A varies one factor at a time around a center point of $\lambda = 4$ requests/s, a 2 s deadline, five edge nodes,

TABLE 3. EDGEINTENT V1 CONDITIONS: TEST AND DEVELOPMENT CASES, INPUT LENGTH PERCENTILES, AND FIRST-PASS AND FINAL YIELD OF THE BLIND-VERIFICATION GATE. DERIVED CONDITIONS ARE BUILT PROGRAMMATICALLY FROM VERIFIED TEXT.

Condition	Kind	N test	N dev	Length p5	Length p50	Length p95	First-pass yield	Final yield
RQ1a								
Base length	derived	300	60	24	38	64	–	–
Padded to 512 tokens	derived	300	60	510	511	512	–	–
Padded to 2,048 tokens	derived	300	60	2046	2047	2048	–	–
Padded to 8,192 tokens	derived	300	60	8190	8191	8192	–	–
Padded to 16,384 tokens	derived	300	60	16382	16383	16384	–	–
RQ1b								
$k = 1$ request per message	derived	300	60	28	42	68	–	–
$k = 2$ requests per message	derived	300	60	66	87	123	–	–
$k = 4$ requests per message	derived	300	60	149	177	223	–	–
$k = 8$ requests per message	derived	300	60	302.9	356	432	–	–
RQ2								
Clean	generated	300	60	24	38	64	1.000	1.000
Code-switched	generated	300	60	30	43	68.1	0.933	1.000
Colloquial	generated	300	60	25	41	69	1.000	1.000
Default-baiting	generated	300	60	33	44	69	0.900	1.000
Key-value	generated	300	60	14	27	53	0.997	1.000
Negation-heavy	generated	300	60	30	45	69.1	1.000	1.000
Noisy	derived	300	60	24	38	66	–	–
Self-revising	generated	300	60	31	46	79.1	0.994	1.000
RQ3								
$F = 4$, low density	generated	300	60	21	37.5	65	0.997	1.000
$F = 4$, medium density	generated	300	60	36	51	72.1	1.000	1.000
$F = 4$, high density	generated	300	60	61	77.5	108	1.000	1.000
$F = 6$, low density	generated	300	60	33	50	76.1	1.000	1.000
$F = 6$, medium density	generated	300	60	46	61	90	0.978	1.000
$F = 6$, high density	generated	300	60	67	81	108	0.983	1.000
$F = 8$, low density	generated	300	60	29	41	77	0.983	1.000
$F = 8$, medium density	generated	300	60	46	65	101.1	0.989	1.000
$F = 8$, high density	generated	300	60	68	87	124	1.000	1.000
RQ4								
$K = 4$ services	generated	300	60	27	46	76	0.975	1.000
$K = 15$ services	generated	300	60	33	52	82.1	0.978	1.000
$K = 64$ services	generated	300	60	38	55	81.1	0.997	1.000
$K = 128$ services	generated	300	60	36	54.5	78	0.978	1.000
$K = 254$ services	generated	300	60	34	51	75.1	0.992	1.000
25% churn at $K = 64$	generated	300	60	35	53	81	0.989	1.000
50% churn at $K = 64$	generated	300	60	35	56	83.1	0.992	1.000

changing text, and caching disabled. The load sweep covers $\lambda \in \{1, 2, 4, 8, 16\}$, the deadline sweep $D \in \{0.5, 1, 2, 4\}$ s, and the topology sweep 5 to 40 edge nodes at $\lambda = 8$. A reuse block crosses changing or repeated descriptions with caching disabled or enabled, and one bursty condition matches the mean rate. The 15 cells each hold 300 arrivals drawn from the clean four-field test split. Live admission events feed modeled execution on the same elapsed-time axis, and no images are processed in Part A.

Part B holds the corresponding request descriptions, image assignments, arrival schedules, and service configuration fixed across all interpreters. Its eight conditions cross steady or bursty arrivals, changing or repeated descriptions, and caching disabled or enabled. Complete interpreter runs are executed sequentially in randomized order. Consequently, matching controls the workload but does not eliminate variation in provider or network state.

The real service uses Tesseract recognition [40] on images from the IIIT5K scene-text dataset [41]. Three workers run identical container images, whereas link emulation gives the second edge node and the cloud node their delays and bandwidths. Separate development images calibrate the scheduler’s service-duration estimates. Evaluation images are selected before recognition results are observed and are not filtered by whether a tier recognizes them correctly. The tiers are execution configurations and do not guarantee recognition quality. Each condition includes 180 OCR

requests and 60 requests for counting or detection, which the testbed does not run and must reject. Changing-text conditions use 180 distinct OCR descriptions, drawn from the clean and the four-field constraint-density test splits. Images and intent descriptions are reused across conditions, while a new service run is still required after an interpretation-cache hit. Deadlines and offered loads are controlled study parameters.

4.4. Correctness and Completion

Exact semantic correctness requires all interpreted fields to equal their reference values. RQ1–RQ4 measure it on every test case, together with the valid-output rate and three field-level failure rates. An unsafe decision permits remote execution where the reference keeps the payload on site. A spurious specification states a value where the reference is unspecified, and a missed constraint leaves a stated requirement unspecified. RQ4 adds top-1 service accuracy on seen and unseen services and the F1 score for detecting unsupported requests.

Operational completion instead requires a supported request to finish execution by its deadline while satisfying the reference service, locality, minimum tier, and mapped priority. Its strict companion additionally requires exact field equality. Part B’s correct completion adds equality between the returned OCR text and the reference text after

canonical Unicode normalization and removal of surrounding whitespace. Case and internal characters are retained. These criteria distinguish interpretation accuracy, execution compliance, and service-output correctness.

For a given backend and condition, let K be the number of supported requests and let C be the number meeting the applicable completion criterion. For $K > 0$, the completion rate is C/K . Because matched conditions share K , their completion counts can also be compared directly. Unsupported requests and their correct rejections are reported separately. Outcome distributions over all arrivals retain both unsupported requests and failures. This separation prevents successful rejections from inflating service completion.

4.5. Latency and API Cost

Decision latency is measured from client call initiation to receipt of the full interpretation response. Admission waiting is accounted for separately in the service timeline. We report medians and empirical 95th percentiles (p95), together with the interquartile range (IQR) as jitter and the probability that a decision exceeds a latency budget τ .

For backend m and request i , let f_i^m be the time when its service response reaches the originating controller. Its full request latency is $T_i^m = f_i^m - a_i$. The full request latency T_i^m includes admission waiting, interpretation, service queueing, transfer, and execution. For a matched pair of Jev-1.13.0 and an LLM L , let $\mathcal{S}$ contain the requests completed correctly by both backends, and let $N = |\mathcal{S}|$. The median paired saving Δ_{paired} is

$$\Delta_{\text{paired}} = \text{median}_{i \in \mathcal{S}} (T_i^L - T_i^{\text{Jev}}). \quad (2)$$

Positive values favor Jev. The percentage reduction of median latency uses a different summary. With M_m denoting the median of T_i^m over $\mathcal{S}$, the percentage reduction R_T is

$$R_T = 100 \left(1 - \frac{M_{\text{Jev}}}{M_L} \right). \quad (3)$$

Equation 2 need not equal $M_L - M_{\text{Jev}}$. Because both summaries are conditional on shared success, we report N and each backend’s completion count alongside them.

For a backend and condition, let A be the provider-reported interpretation fees in United States dollars (USD) and let c be API cost per completion. Using the corresponding completion count C , we define

$$c = \frac{A}{C}, \quad C > 0. \quad (4)$$

The numerator includes all interpretation calls for that condition, including unsupported requests, unsuccessful jobs, and calls ending after their deadline. It excludes warmups. RQ1–RQ4 use exactly correct decisions for C and report c per 1,000 correct decisions. Part A uses operational completion and Part B correct OCR completion. Cost is undefined when $C = 0$. Relative cost reductions use the same ratio as Equation 3, with c replacing M . These fees exclude computation, communication infrastructure, energy,

and maintenance. Because self-hosted models carry no API fee, we report energy per decision for them, integrated from the accelerator power trace.

4.6. Statistical Analysis

Hypotheses H1–H8 and their decision rules were fixed before the runs. Within each RQ, the unit is the test case, and interpreters are compared on the same cases. Confidence intervals come from 10,000 case bootstrap resamples. Paired proportions use exact McNemar tests, latencies Wilcoxon signed-rank tests, and the six-way comparison of unsafe decisions Cochran’s Q . Holm’s method controls the error rate within each hypothesis family, and a contrast counts as resolved when its 95% interval excludes the null value and its adjusted p is below 0.05. Latency contrasts are confirmatory only within a deployment class, hosted against hosted and self-hosted against self-hosted. Because RQ5 arrivals share queues and caches, its intervals use a block bootstrap over 30 contiguous arrivals. Cells in which an interpreter returns fewer than half valid outputs are still tested and are flagged. Eight recorded deviations from the protocol include the GLM-5.3-Flash reasoning setting, the 15-service catalog level chosen for SemIf-Qwen3.5-4B’s option limit, a single run per case, and the corpus generators and verifier. The remaining deviations cover the accelerator for the self-hosted models, shared reading rules in the field instructions, one availability re-run for GLM-5.3-Flash, and the classifier training set.

4.7. Implementation and Settings

Interpretation backends. The interpretation-layer runs were executed on September 24–25, 2026. Jev uses OpenRouter’s Decisions endpoint with identifier typesafe/jev-1.13. Responses identify jev-1.13-20260917 and provider TypeSafe. One native Choice question per contract field returns the intent fields in one request. DeepSeek-V4.1-Flash and GLM-5.3-Flash are pinned to Together and Qwen3.8-Flash to Alibaba, its only endpoint. All three have provider fallback disabled and data collection denied. They return a strict JSON object with temperature zero. Reasoning is disabled and recorded reasoning-token counts are zero for DeepSeek-V4.1-Flash and Qwen3.8-Flash. Because no GLM-5.3-Flash endpoint accepts disabled reasoning, the model runs at the lowest setting. Its reasoning tokens count toward its output tokens, latency, and cost. Calls have a 30 s timeout, no retries, and no caching.

SemIf-Qwen3.5-4B (SemIf-OpenJev commit 23cf1f39 over Qwen/Qwen3.5-4B revision 851bf6e8), Laya (revision bc76315b), and Qwen3.5-4B-JSON (the same Qwen3.5-4B weights with the LLM prompt and greedy decoding) run one at a time on an NVIDIA H100 NVL. These models use PyTorch 2.13.0 and Transformers 5.17.0 and execute in time blocks separate from the hosted runs. The fixed rule parser uses keywords and regular expressions without tuning on evaluation wording. The MiniLM-Reranker scores

the cosine similarity between a request and each service description and abstains below a threshold of 0.3691, calibrated on the development split. The DistilBERT classifier heads are fine-tuned on the development split plus one description example per service. The fine-tuning fixes a maximum length of 128 tokens, learning rate 5×10^{-5} , batch size 16, ten epochs, and seed 20260924 without a search.

Admission and execution controls. Both parts of RQ5 use four concurrent interpretation slots and an admission queue of capacity 32, with no automatic retries or repair calls. Connections are warmed before measurement. Each condition starts with an empty cache keyed by normalized full request text and extraction policy. The controller checks it before queuing and after obtaining a slot, but concurrent misses are not coalesced. Self-hosted interpreters run on the same host as the controller, in their own time blocks.

Each worker has one nonpreemptive queue. Urgent and ordinary jobs have priority values zero and one, respectively, and equal-priority jobs follow enqueue order. The controller verifies the returned node, tier, priority, and image identity. A timed-out OCR process is terminated before capacity is released.

Part A parameters. Steady traffic follows a Poisson process. Bursty traffic alternates 0.5 and 8 requests/s in 20 s segments. The execution model has N edge nodes and one cloud node, with payloads of 0.25–2 MB. Local, other-edge, and cloud links have propagation delays of 2, 20, and 60 ms and bandwidths of 1000, 100, and 50 Mbit/s, respectively. Transfer over these links also includes serialization. Base durations are 40 ms for counting, 80 ms for detection, and 60 ms for OCR. The high tier multiplies these by 1.8. Edge node factors are spread over 1.0–1.3, and the cloud factor is 0.65. Each node has one priority worker. Repeated-text cells use eight recurring descriptions.

Part B parameters and scoring. The three workers run the same container image with one Tesseract build and English recognition in single-word mode. Standard and high tiers use `tessdata_fast` and `tessdata_best`, respectively, with identical weights across nodes. The second edge node’s link is emulated at 10 ms one-way delay and 100 Mbit/s, and the cloud node’s at 30 ms and 50 Mbit/s. A seeded split of IIIT5K identifiers selects calibration and test images before recognition outcomes are known. Median request–response durations on 40 calibration images are 52 and 59 ms on the local node for the standard and high tiers. The corresponding durations are 76 and 89 ms on the second edge node and 128 and 137 ms on the cloud node. Steady arrivals average 2 requests/s, and bursty arrivals follow the Part A pattern. Repeated-text conditions cycle six OCR and two unsupported descriptions. The request deadline is 2 s, and each run holds 240 arrivals.

OCR comparison uses case-preserving character annotations without a recognition lexicon, normalizes Unicode to Normalization Form C (NFC), and strips surrounding whitespace. Each request is evaluated on its recorded timing and output, and cached decisions do not replace service execution.

5. Results

Table 12 summarizes every pre-registered contrast of H1–H5, with its estimate, 95% confidence interval, and Holm-adjusted p . The subsections below examine these contrasts per RQ. H6–H8 are tested in Section 5.6 under the same criterion, with the Holm adjustment applied within each part.

5.1. RQ1: Input Scale

Longer inputs slow every interpreter, and Jev-1.13.0 least. Its median decision latency grows from 0.274 s at the base length to 0.418 s at 16,384 tokens (Fig. 3 and Table 4). Over the same range, DeepSeek-V4.1-Flash grows from 0.378 to 0.638 s, GLM-5.3-Flash from 0.646 to 1.588 s, and Qwen3.8-Flash from 1.774 to 2.920 s. Jev-1.13.0 is faster than each LLM at every length, and all 15 hosted contrasts resolve (H1). Its growth from base to 16,384 tokens is 0.90, 0.62, and 0.93 times that of DeepSeek-V4.1-Flash, GLM-5.3-Flash, and Qwen3.8-Flash, respectively. Exact match stays at 0.910–0.940 for Jev-1.13.0 and 0.973–0.990 for DeepSeek-V4.1-Flash, whereas Qwen3.8-Flash drops to 0.763 at 8,192 tokens.

The self-hosted pair shows the opposite growth pattern. SemIf-Qwen3.5-4B answers 0.39–0.65 s faster than Qwen3.5-4B-JSON at every length, yet its latency grows from 0.124 to 1.159 s, 4.68 times the growth of Qwen3.5-4B-JSON. At these weights, input length costs the readout model more than the generator.

Bundling changes the picture more sharply. Placing eight requests in one message (Table 5) raises Jev-1.13.0’s message latency from 0.272 to 0.290 s, while DeepSeek-V4.1-Flash rises from 0.379 to 0.819 s, GLM-5.3-Flash from 0.583 to 1.560 s, and Qwen3.8-Flash from 1.302 to 3.785 s. Jev-1.13.0’s per-request latency ratio from one to eight requests is 0.49, 0.40, and 0.37 times the LLMs’ ratios. All three contrasts resolve (H2), as does the self-hosted contrast (0.53). Request-level exact match stays at 0.948–0.957 for Jev-1.13.0 and 0.982–0.987 for DeepSeek-V4.1-Flash, while SemIf-Qwen3.5-4B falls from 0.593 to 0.301 and Laya from 0.080 to zero.

Takeaway **Input scale.** Irrelevant context and bundled requests leave Jev-1.13.0’s decision time within 0.27–0.42 s, while every hosted LLM’s time at least doubles with eight bundled requests. The price is 2.8–6.7 points of exact match relative to DeepSeek-V4.1-Flash.

5.2. RQ2: Input Quality

Most wording changes cost little accuracy. Leaving out the default-baiting condition, Jev-1.13.0’s exact match stays at 0.920–0.950 across the other seven conditions and DeepSeek-V4.1-Flash’s at 0.970–0.990 (Fig. 4 and Table 6). Default-baiting wording, which hints at values that the request never states, lowers every interpreter. GLM-5.3-Flash keeps 0.653, Jev-1.13.0 0.610, DeepSeek-V4.1-Flash

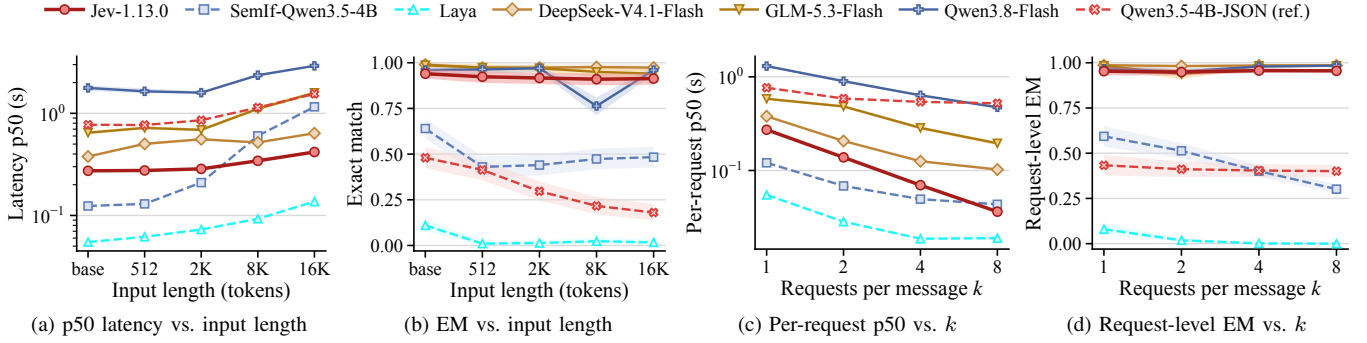


Figure 3. RQ1, input scale. (a) Median decision latency and (b) exact match (EM) as irrelevant context pads each request to 16,384 tokens. (c) Per-request median latency and (d) request-level EM when k requests share one message. Bands are 95% bootstrap confidence intervals; latency axes are logarithmic.

TABLE 4. RQ1A, INPUT LENGTH. EM WITH 95% CONFIDENCE INTERVAL, VALID-OUTPUT RATE, MACRO FIELD ACCURACY, DECISION LATENCY PERCENTILES, OUTPUT TOKENS, API FEES PER 1,000 CORRECT DECISIONS, AND ENERGY PER DECISION. BEST VALUE PER BLOCK IN BOLD.

Model	EM [95% CI] ↑	Valid ↑	Macro acc. ↑	p50 (s) ↓	p95 (s) ↓	p99 (s) ↓	Output tokens	USD per 1k correct ↓	Energy (J/dec.) ↓
Base length									
Jev-1.13.0	0.940 [0.913, 0.967]	1.000	0.985	0.274	0.381	0.536	181	0.03650	—
SemIf-Qwen3.5-4B	0.640 [0.587, 0.693]	1.000	0.902	0.124	0.296	0.323	0	—	23.7
Laya	0.110 [0.077, 0.147]	1.000	0.671	0.055	0.222	0.242	0	—	8.4
DeepSeek-V4.1-Flash	0.990 [0.977, 1.000]	1.000	0.998	0.378	1.054	4.933	26	0.09051	—
GLM-5.3-Flash	0.987 [0.973, 0.997]	1.000	0.997	0.646	2.685	3.363	36	0.04048	—
Qwen3.8-Flash	0.960 [0.937, 0.980]	0.997	0.988	1.774	3.668	5.774	30	0.03797	—
Qwen3.5-4B-JSON (ref.)	0.480 [0.423, 0.537]	1.000	0.854	0.774	0.868	1.036	30	—	127.2
Padded to 512 tokens									
Jev-1.13.0	0.923 [0.890, 0.953]	1.000	0.981	0.276	0.359	0.505	181	0.06585	—
SemIf-Qwen3.5-4B	0.430 [0.373, 0.487]	1.000	0.805	0.130	0.231	0.290	0	—	30.9
Laya	0.010 [0.000, 0.023]	1.000	0.525	0.062	0.150	0.191	0	—	11.6
DeepSeek-V4.1-Flash	0.973 [0.953, 0.990]	1.000	0.993	0.503	0.883	2.113	25	0.2534	—
GLM-5.3-Flash	0.973 [0.953, 0.990]	1.000	0.993	0.720	2.469	9.620	26	0.1146	—
Qwen3.8-Flash	0.963 [0.940, 0.983]	0.997	0.988	1.643	4.566	7.140	36	0.1462	—
Qwen3.5-4B-JSON (ref.)	0.413 [0.357, 0.467]	1.000	0.818	0.768	0.872	1.218	30	—	130.5
Padded to 2,048 tokens									
Jev-1.13.0	0.917 [0.883, 0.947]	1.000	0.979	0.286	0.404	0.522	181	0.1606	—
SemIf-Qwen3.5-4B	0.440 [0.383, 0.497]	1.000	0.819	0.210	0.334	0.359	0	—	61.6
Laya	0.013 [0.003, 0.027]	1.000	0.380	0.073	0.074	0.075	0	—	14.4
DeepSeek-V4.1-Flash	0.973 [0.953, 0.990]	1.000	0.993	0.557	1.076	19.198	25	0.7535	—
GLM-5.3-Flash	0.970 [0.950, 0.987]	1.000	0.992	0.689	2.518	3.177	26	0.3642	—
Qwen3.8-Flash	0.970 [0.950, 0.987]	1.000	0.993	1.594	2.408	4.261	40	0.4526	—
Qwen3.5-4B-JSON (ref.)	0.297 [0.247, 0.350]	1.000	0.765	0.857	0.981	1.545	31	—	162.0
Padded to 8,192 tokens									
Jev-1.13.0	0.910 [0.877, 0.940]	1.000	0.977	0.343	0.525	0.839	181	0.5417	—
SemIf-Qwen3.5-4B	0.473 [0.417, 0.530]	1.000	0.823	0.602	0.718	0.748	0	—	194.9
Laya	0.023 [0.007, 0.043]	1.000	0.395	0.093	0.094	0.095	0	—	16.3
DeepSeek-V4.1-Flash	0.977 [0.957, 0.993]	1.000	0.993	0.519	0.785	1.232	25	2.753	—
GLM-5.3-Flash	0.950 [0.923, 0.973]	0.993	0.981	1.107	5.631	29.708	26	1.384	—
Qwen3.8-Flash	0.763 [0.713, 0.810]	1.000	0.909	2.364	4.791	8.870	38	2.144	—
Qwen3.5-4B-JSON (ref.)	0.217 [0.170, 0.267]	1.000	0.714	1.140	1.206	1.859	31	—	279.8
Padded to 16,384 tokens									
Jev-1.13.0	0.913 [0.880, 0.943]	1.000	0.978	0.418	0.523	1.198	181	1.044	—
SemIf-Qwen3.5-4B	0.483 [0.427, 0.540]	1.000	0.829	1.159	1.238	1.288	0	—	378.8
Laya	0.017 [0.003, 0.033]	1.000	0.392	0.137	0.140	0.142	0	—	20.4
DeepSeek-V4.1-Flash	0.973 [0.953, 0.990]	1.000	0.993	0.638	1.023	1.183	25	5.455	—
GLM-5.3-Flash	0.940 [0.913, 0.967]	0.980	0.968	1.588	3.939	12.754	26	2.727	—
Qwen3.8-Flash	0.960 [0.937, 0.980]	1.000	0.990	2.920	4.274	5.400	38	3.373	—
Qwen3.5-4B-JSON (ref.)	0.180 [0.137, 0.227]	1.000	0.694	1.550	1.619	2.384	31	—	444.7

0.530, and Qwen3.8-Flash 0.470. The failures are spurious specifications, values that the reference leaves unspecified, at rates of 0.264–0.428 for the four hosted interpreters.

Unsafe decisions differ across the six interpreters in all eight conditions (Cochran’s Q , H3). The difference comes from Laya, whose unsafe rate is 0.033–0.133 and exceeds Jev-1.13.0’s in seven of eight paired contrasts (Table 7). Jev-1.13.0 makes no unsafe decision in seven conditions and 4.3% under default-baiting, and the hosted LLMs stay at or below 5.0%. None of the paired contrasts between Jev-1.13.0 and an LLM or SemIf-Qwen3.5-4B resolves.

Takeaway **Input quality.** Because all four hosted interpreters keep unsafe placement at or below 5% under every wording, wording robustness does not separate them. Default-baiting is the shared weak point, and there Jev-1.13.0 ranks second of the six.

5.3. RQ3: Task Difficulty

Contract size is where the decision model pays. With four fields, Jev-1.13.0 reaches an exact match of 0.923–0.967, close to the LLMs. With six fields it falls to 0.780–

TABLE 5. RQ1B, BUNDLED REQUESTS. REQUEST-LEVEL EM, SHARE OF MESSAGES WITH EVERY REQUEST CORRECT, PER-REQUEST AND PER-MESSAGE LATENCY, API FEES, AND ENERGY FOR k REQUESTS PER MESSAGE.

Model	Request EM [95% CI] ↑	Message all-correct ↑	Per-request p50 (s) ↓	Message p50 (s) ↓	Message p95 (s) ↓	USD per 1k correct ↓	Energy (J/dec.) ↓
$k = 1$ request per message							
Jev-1.13.0	0.953 [0.930, 0.977]	0.953	0.272	0.272	0.372	0.03617	–
SemIf-Qwen3.5-4B	0.593 [0.537, 0.650]	0.593	0.121	0.121	0.121	–	22.3
Laya	0.080 [0.050, 0.113]	0.080	0.055	0.055	0.056	–	6.6
DeepSeek-V4.1-Flash	0.987 [0.973, 0.997]	0.987	0.379	0.379	0.552	0.1113	–
GLM-5.3-Flash	0.983 [0.967, 0.997]	0.983	0.583	0.583	0.817	0.03997	–
Qwen3.8-Flash	0.967 [0.943, 0.987]	0.967	1.302	1.302	2.221	0.05417	–
Qwen3.5-4B-JSON (ref.)	0.433 [0.377, 0.490]	0.433	0.765	0.765	0.794	–	121.8
$k = 2$ requests per message							
Jev-1.13.0	0.948 [0.930, 0.965]	0.897	0.138	0.276	0.359	0.06665	–
SemIf-Qwen3.5-4B	0.513 [0.472, 0.555]	0.283	0.068	0.137	0.312	–	34.0
Laya	0.018 [0.008, 0.030]	0.000	0.028	0.057	0.207	–	10.5
DeepSeek-V4.1-Flash	0.982 [0.970, 0.992]	0.963	0.206	0.413	0.626	0.1757	–
GLM-5.3-Flash	0.937 [0.910, 0.960]	0.920	0.483	0.966	4.237	0.07183	–
Qwen3.8-Flash	0.952 [0.935, 0.968]	0.903	0.898	1.796	2.952	0.09168	–
Qwen3.5-4B-JSON (ref.)	0.412 [0.368, 0.455]	0.223	0.587	1.173	1.277	–	194.4
$k = 4$ requests per message							
Jev-1.13.0	0.957 [0.944, 0.968]	0.850	0.070	0.278	0.358	0.1278	–
SemIf-Qwen3.5-4B	0.401 [0.369, 0.433]	0.047	0.049	0.198	0.330	–	57.1
Laya	0.002 [0.000, 0.004]	0.000	0.019	0.075	0.197	–	17.2
DeepSeek-V4.1-Flash	0.985 [0.978, 0.992]	0.940	0.125	0.502	0.687	0.2669	–
GLM-5.3-Flash	0.983 [0.974, 0.990]	0.933	0.285	1.140	2.113	0.1207	–
Qwen3.8-Flash	0.979 [0.969, 0.988]	0.927	0.635	2.540	3.445	0.1365	–
Qwen3.5-4B-JSON (ref.)	0.404 [0.368, 0.440]	0.053	0.541	2.162	2.286	–	358.3
$k = 8$ requests per message							
Jev-1.13.0	0.955 [0.946, 0.964]	0.717	0.036	0.290	0.375	0.2881	–
SemIf-Qwen3.5-4B	0.301 [0.278, 0.325]	0.003	0.044	0.348	0.773	–	109.6
Laya	0.000 [0.000, 0.000]	0.000	0.019	0.152	0.225	–	40.7
DeepSeek-V4.1-Flash	0.985 [0.980, 0.990]	0.887	0.102	0.819	1.720	0.4706	–
GLM-5.3-Flash	0.983 [0.978, 0.988]	0.867	0.195	1.560	2.884	0.2264	–
Qwen3.8-Flash	0.984 [0.978, 0.989]	0.880	0.473	3.785	5.499	0.2288	–
Qwen3.5-4B-JSON (ref.)	0.401 [0.366, 0.435]	0.020	0.522	4.174	6.142	–	731.9

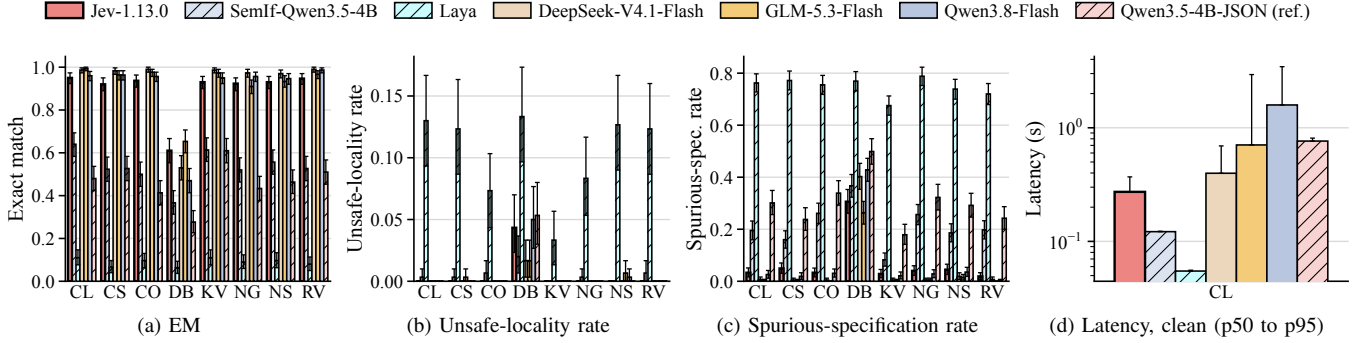


Figure 4. RQ2, input quality, across eight wording conditions that keep the same label tuples. (a) EM, (b) unsafe-locality rate, (c) spurious-specification rate, with 95% confidence intervals; (d) decision latency on clean requests, bars at the median with whiskers to p95. Conditions: CL clean, CS code-switched, CO colloquial, DB default-baiting, KV key-value, NG negation-heavy, NS noisy, RV self-revising.

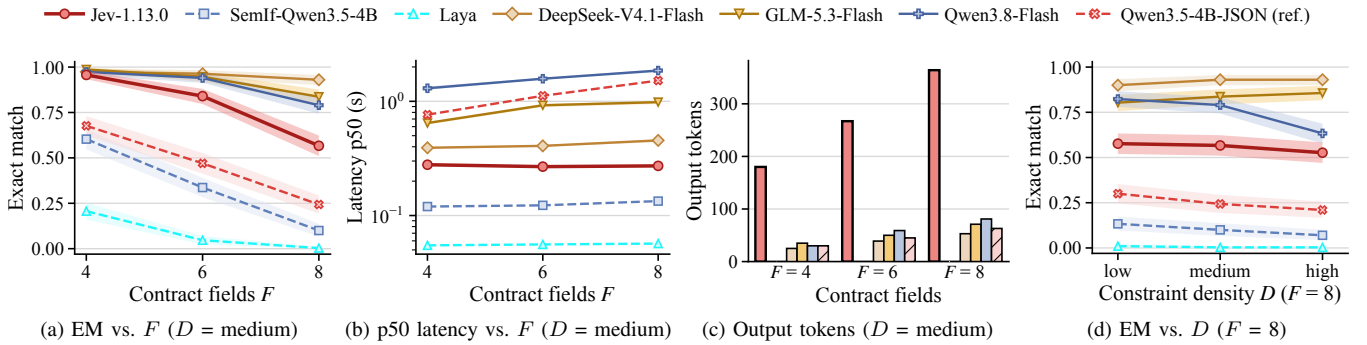


Figure 5. RQ3, task difficulty. (a) EM, (b) median decision latency, and (c) median output tokens as the contract grows from four to eight fields at medium constraint density; (d) EM across constraint density with eight fields.

TABLE 6. RQ2, INPUT QUALITY. QUALITY, SAFETY, AND LATENCY OF EVERY INTERPRETER UNDER EACH WORDING CONDITION.

Model	EM [95% CI] ↑	Macro acc. ↑	Valid ↑	Unsafe ↓	Spurious ↓	Missed ↓	p50 (s) ↓	p95 (s) ↓
Clean								
Jev-1.13.0	0.950 [0.923, 0.973]	0.988	1.000	0.000	0.033	0.001	0.273	0.370
SemIf-Qwen3.5-4B	0.640 [0.583, 0.693]	0.902	1.000	0.003	0.195	0.031	0.122	0.123
Laya	0.110 [0.077, 0.147]	0.671	1.000	0.130	0.762	0.005	0.055	0.056
DeepSeek-V4.1-Flash	0.987 [0.973, 0.997]	0.997	1.000	0.000	0.007	0.001	0.398	0.692
GLM-5.3-Flash	0.993 [0.983, 1.000]	0.998	1.000	0.000	0.002	0.001	0.706	2.940
Qwen3.8-Flash	0.960 [0.937, 0.980]	0.988	0.997	0.000	0.026	0.004	1.588	3.450
Qwen3.5-4B-JSON (ref.)	0.480 [0.423, 0.537]	0.854	1.000	0.000	0.301	0.026	0.764	0.811
Code-switched								
Jev-1.13.0	0.920 [0.890, 0.950]	0.980	1.000	0.000	0.049	0.003	0.278	0.366
SemIf-Qwen3.5-4B	0.523 [0.467, 0.580]	0.865	0.997	0.003	0.160	0.076	0.121	0.122
Laya	0.067 [0.040, 0.097]	0.605	1.000	0.123	0.772	0.014	0.055	0.056
DeepSeek-V4.1-Flash	0.983 [0.967, 0.997]	0.996	1.000	0.000	0.005	0.004	0.490	1.213
GLM-5.3-Flash	0.963 [0.940, 0.983]	0.991	1.000	0.003	0.002	0.010	0.729	3.115
Qwen3.8-Flash	0.963 [0.940, 0.983]	0.991	1.000	0.000	0.019	0.003	1.416	2.726
Qwen3.5-4B-JSON (ref.)	0.527 [0.470, 0.583]	0.859	1.000	0.000	0.238	0.027	0.765	0.813
Colloquial								
Jev-1.13.0	0.937 [0.907, 0.963]	0.983	1.000	0.000	0.033	0.008	0.268	0.372
SemIf-Qwen3.5-4B	0.500 [0.443, 0.557]	0.853	1.000	0.007	0.261	0.043	0.121	0.122
Laya	0.097 [0.063, 0.130]	0.656	1.000	0.073	0.755	0.009	0.055	0.055
DeepSeek-V4.1-Flash	0.990 [0.977, 1.000]	0.998	1.000	0.000	0.005	0.001	0.392	0.629
GLM-5.3-Flash	0.977 [0.957, 0.990]	0.994	1.000	0.000	0.000	0.009	0.739	1.748
Qwen3.8-Flash	0.957 [0.933, 0.977]	0.989	1.000	0.000	0.031	0.000	1.790	4.053
Qwen3.5-4B-JSON (ref.)	0.413 [0.357, 0.470]	0.833	1.000	0.000	0.339	0.035	0.763	0.809
Default-baiting								
Jev-1.13.0	0.610 [0.553, 0.667]	0.885	1.000	0.043	0.306	0.009	0.262	0.351
SemIf-Qwen3.5-4B	0.367 [0.313, 0.423]	0.792	1.000	0.020	0.367	0.068	0.122	0.122
Laya	0.063 [0.037, 0.093]	0.612	1.000	0.133	0.769	0.021	0.055	0.056
DeepSeek-V4.1-Flash	0.530 [0.473, 0.587]	0.858	1.000	0.017	0.402	0.000	0.365	0.486
GLM-5.3-Flash	0.653 [0.600, 0.707]	0.896	1.000	0.017	0.264	0.017	0.820	3.251
Qwen3.8-Flash	0.470 [0.413, 0.527]	0.848	1.000	0.050	0.428	0.000	1.299	2.073
Qwen3.5-4B-JSON (ref.)	0.277 [0.227, 0.330]	0.777	1.000	0.053	0.499	0.018	0.762	0.789
Key-value								
Jev-1.13.0	0.930 [0.900, 0.957]	0.982	1.000	0.000	0.028	0.012	0.278	0.403
SemIf-Qwen3.5-4B	0.613 [0.560, 0.670]	0.889	1.000	0.000	0.082	0.059	0.121	0.122
Laya	0.110 [0.077, 0.147]	0.610	1.000	0.033	0.675	0.040	0.055	0.055
DeepSeek-V4.1-Flash	0.987 [0.973, 0.997]	0.997	1.000	0.000	0.005	0.003	0.401	0.667
GLM-5.3-Flash	0.973 [0.953, 0.990]	0.993	1.000	0.000	0.002	0.005	0.907	3.082
Qwen3.8-Flash	0.950 [0.923, 0.973]	0.988	1.000	0.000	0.021	0.003	1.278	2.125
Qwen3.5-4B-JSON (ref.)	0.610 [0.553, 0.667]	0.887	1.000	0.000	0.179	0.023	0.763	0.809
Negation-heavy								
Jev-1.13.0	0.923 [0.890, 0.950]	0.981	1.000	0.000	0.040	0.008	0.286	0.437
SemIf-Qwen3.5-4B	0.520 [0.463, 0.577]	0.860	1.000	0.003	0.256	0.062	0.121	0.122
Laya	0.090 [0.060, 0.123]	0.634	1.000	0.083	0.788	0.008	0.055	0.056
DeepSeek-V4.1-Flash	0.973 [0.953, 0.990]	0.993	1.000	0.000	0.005	0.008	0.377	0.546
GLM-5.3-Flash	0.910 [0.877, 0.940]	0.950	0.963	0.000	0.005	0.054	0.809	3.969
Qwen3.8-Flash	0.957 [0.933, 0.977]	0.989	1.000	0.000	0.028	0.001	1.522	2.716
Qwen3.5-4B-JSON (ref.)	0.433 [0.377, 0.490]	0.823	1.000	0.000	0.322	0.036	0.762	0.811
Noisy								
Jev-1.13.0	0.930 [0.900, 0.957]	0.983	1.000	0.000	0.045	0.003	0.285	0.399
SemIf-Qwen3.5-4B	0.557 [0.500, 0.613]	0.877	1.000	0.000	0.186	0.053	0.121	0.122
Laya	0.097 [0.063, 0.133]	0.645	1.000	0.127	0.739	0.014	0.055	0.056
DeepSeek-V4.1-Flash	0.970 [0.950, 0.987]	0.993	1.000	0.000	0.019	0.001	0.368	0.604
GLM-5.3-Flash	0.933 [0.907, 0.960]	0.973	0.973	0.007	0.014	0.032	0.671	2.728
Qwen3.8-Flash	0.947 [0.920, 0.970]	0.987	1.000	0.003	0.035	0.000	1.535	2.750
Qwen3.5-4B-JSON (ref.)	0.463 [0.410, 0.520]	0.838	1.000	0.000	0.292	0.035	0.763	0.811
Self-revising								
Jev-1.13.0	0.947 [0.920, 0.970]	0.987	1.000	0.000	0.019	0.006	0.277	0.394
SemIf-Qwen3.5-4B	0.527 [0.470, 0.583]	0.861	1.000	0.007	0.198	0.034	0.121	0.122
Laya	0.080 [0.050, 0.113]	0.593	1.000	0.123	0.720	0.012	0.055	0.056
DeepSeek-V4.1-Flash	0.990 [0.977, 1.000]	0.998	1.000	0.000	0.007	0.000	0.398	0.637
GLM-5.3-Flash	0.967 [0.947, 0.983]	0.992	1.000	0.000	0.000	0.008	0.871	2.816
Qwen3.8-Flash	0.987 [0.973, 0.997]	0.997	1.000	0.000	0.002	0.000	1.456	2.855
Qwen3.5-4B-JSON (ref.)	0.510 [0.453, 0.567]	0.850	1.000	0.000	0.242	0.023	0.765	0.983

0.840, and with eight to 0.527–0.577, while DeepSeek-V4.1-Flash keeps 0.900–0.930 (Fig. 5 and Table 8). Constraint density moves exact match much less than the number of fields.

The LLMs’ median output grows by 28, 36, and 43 tokens from four to eight fields, and all three contrasts resolve (H4). Their latency grows with it and spans 0.372–0.483 s over the nine cells for DeepSeek-V4.1-Flash and 1.275–2.092 s for Qwen3.8-Flash. Jev-1.13.0’s latency stays at 0.264–0.280 s in every cell. Its growth from four to

eight fields is 0.83, 0.64, and 0.65 times that of the three LLMs, and 0.56 for SemIf-Qwen3.5-4B against Qwen3.5-4B-JSON. These four contrasts all resolve.

Takeaway Task difficulty. Each added field costs an LLM output tokens and time but leaves Jev-1.13.0’s time unchanged. At eight fields the trade reverses. DeepSeek-V4.1-Flash is then 32.3–40.3 points more accurate for about 0.2 s of additional latency.

TABLE 7. RQ2 PAIRED CONTRASTS AGAINST JEV-1.13.0 PER WORDING CONDITION: DIFFERENCES IN EM AND IN THE UNSAFE-LOCALITY RATE WITH 95% CONFIDENCE INTERVALS, EXACT McNEMAR p , AND THE HOLM-ADJUSTED p OF H3.

Contrast	EM diff. [95% CI]	EM McNemar p	Unsafe diff. [95% CI]	Unsafe McNemar p	Unsafe Holm p (H3)	Verdict
Clean						
SemIf-Qwen3.5-4B — Jev-1.13.0	−0.310 [−0.367, −0.253]	8.4e-23	0.003 [0.000, 0.010]	1	1	not resolved
Laya — Jev-1.13.0	−0.840 [−0.880, −0.797]	2.8e-76	0.130 [0.093, 0.170]	3.6e-12	1.5e-10	resolved
DeepSeek-V4.1-Flash — Jev-1.13.0	0.037 [0.013, 0.063]	0.0074	0.000 [0.000, 0.000]	1	1	not resolved
GLM-5.3-Flash — Jev-1.13.0	0.043 [0.017, 0.070]	0.0023	0.000 [0.000, 0.000]	1	1	not resolved
Qwen3.8-Flash — Jev-1.13.0	0.010 [−0.017, 0.037]	0.63	0.000 [0.000, 0.000]	1	1	not resolved
Code-switched						
SemIf-Qwen3.5-4B — Jev-1.13.0	−0.397 [−0.457, −0.337]	2.4e-29	0.003 [0.000, 0.010]	1	1	not resolved
Laya — Jev-1.13.0	−0.853 [−0.893, −0.810]	1.7e-77	0.123 [0.087, 0.160]	1.5e-11	5.5e-10	resolved
DeepSeek-V4.1-Flash — Jev-1.13.0	0.063 [0.033, 0.097]	1.6e-4	0.000 [0.000, 0.000]	1	1	not resolved
GLM-5.3-Flash — Jev-1.13.0	0.043 [0.007, 0.083]	0.035	0.003 [0.000, 0.010]	1	1	not resolved
Qwen3.8-Flash — Jev-1.13.0	0.043 [0.010, 0.077]	0.019	0.000 [0.000, 0.000]	1	1	not resolved
Colloquial						
SemIf-Qwen3.5-4B — Jev-1.13.0	−0.437 [−0.493, −0.380]	2.5e-38	0.007 [0.000, 0.017]	0.5	1	not resolved
Laya — Jev-1.13.0	−0.840 [−0.880, −0.797]	1.8e-74	0.073 [0.047, 0.103]	4.8e-7	1.7e-5	resolved
DeepSeek-V4.1-Flash — Jev-1.13.0	0.053 [0.027, 0.083]	1.4e-4	0.000 [0.000, 0.000]	1	1	not resolved
GLM-5.3-Flash — Jev-1.13.0	0.040 [0.010, 0.073]	0.023	0.000 [0.000, 0.000]	1	1	not resolved
Qwen3.8-Flash — Jev-1.13.0	0.020 [−0.013, 0.057]	0.34	0.000 [0.000, 0.000]	1	1	not resolved
Default-baiting						
SemIf-Qwen3.5-4B — Jev-1.13.0	−0.243 [−0.303, −0.180]	3.5e-13	−0.023 [−0.047, 0.000]	0.092	1	not resolved
Laya — Jev-1.13.0	−0.547 [−0.607, −0.487]	1.2e-44	0.090 [0.050, 0.133]	4.2e-5	0.0014	resolved
DeepSeek-V4.1-Flash — Jev-1.13.0	−0.080 [−0.137, −0.023]	0.0071	−0.027 [−0.050, −0.007]	0.039	1	not resolved
GLM-5.3-Flash — Jev-1.13.0	0.043 [−0.007, 0.093]	0.12	−0.027 [−0.050, −0.007]	0.039	1	not resolved
Qwen3.8-Flash — Jev-1.13.0	−0.140 [−0.197, −0.083]	3.7e-6	0.007 [−0.017, 0.030]	0.79	1	not resolved
Key-value						
SemIf-Qwen3.5-4B — Jev-1.13.0	−0.317 [−0.373, −0.260]	5.0e-24	0.000 [0.000, 0.000]	1	1	not resolved
Laya — Jev-1.13.0	−0.820 [−0.863, −0.773]	1.1e-72	0.033 [0.013, 0.057]	0.002	0.064	not resolved
DeepSeek-V4.1-Flash — Jev-1.13.0	0.057 [0.030, 0.087]	7.6e-5	0.000 [0.000, 0.000]	1	1	not resolved
GLM-5.3-Flash — Jev-1.13.0	0.043 [0.013, 0.073]	0.011	0.000 [0.000, 0.000]	1	1	not resolved
Qwen3.8-Flash — Jev-1.13.0	0.020 [−0.013, 0.053]	0.31	0.000 [0.000, 0.000]	1	1	not resolved
Negation-heavy						
SemIf-Qwen3.5-4B — Jev-1.13.0	−0.403 [−0.463, −0.347]	4.0e-33	0.003 [0.000, 0.010]	1	1	not resolved
Laya — Jev-1.13.0	−0.833 [−0.873, −0.793]	1.1e-75	0.083 [0.053, 0.117]	6.0e-8	2.1e-6	resolved
DeepSeek-V4.1-Flash — Jev-1.13.0	0.050 [0.017, 0.083]	0.0059	0.000 [0.000, 0.000]	1	1	not resolved
GLM-5.3-Flash — Jev-1.13.0	−0.013 [−0.057, 0.030]	0.64	0.000 [0.000, 0.000]	1	1	not resolved
Qwen3.8-Flash — Jev-1.13.0	0.033 [−0.003, 0.070]	0.099	0.000 [0.000, 0.000]	1	1	not resolved
Noisy						
SemIf-Qwen3.5-4B — Jev-1.13.0	−0.373 [−0.433, −0.317]	1.3e-29	0.000 [0.000, 0.000]	1	1	not resolved
Laya — Jev-1.13.0	−0.833 [−0.873, −0.790]	1.1e-75	0.127 [0.090, 0.163]	7.3e-12	2.8e-10	resolved
DeepSeek-V4.1-Flash — Jev-1.13.0	0.040 [0.010, 0.073]	0.023	0.000 [0.000, 0.000]	1	1	not resolved
GLM-5.3-Flash — Jev-1.13.0	0.003 [−0.033, 0.040]	1	0.007 [0.000, 0.017]	0.5	1	not resolved
Qwen3.8-Flash — Jev-1.13.0	0.017 [−0.017, 0.050]	0.44	0.003 [0.000, 0.010]	1	1	not resolved
Self-revising						
SemIf-Qwen3.5-4B — Jev-1.13.0	−0.420 [−0.480, −0.363]	1.4e-34	0.007 [0.000, 0.017]	0.5	1	not resolved
Laya — Jev-1.13.0	−0.867 [−0.907, −0.827]	7.1e-77	0.123 [0.087, 0.163]	1.5e-11	5.5e-10	resolved
DeepSeek-V4.1-Flash — Jev-1.13.0	0.043 [0.017, 0.070]	0.0023	0.000 [0.000, 0.000]	1	1	not resolved
GLM-5.3-Flash — Jev-1.13.0	0.020 [−0.007, 0.050]	0.24	0.000 [0.000, 0.000]	1	1	not resolved
Qwen3.8-Flash — Jev-1.13.0	0.040 [0.013, 0.067]	0.0075	0.000 [0.000, 0.000]	1	1	not resolved

5.4. RQ4: Dynamic Service Catalog

Interpreters that receive the catalog with each request cope with both its size and its churn. Jev-1.13.0 and the three LLMs name known services with a top-1 accuracy of at least 0.985 at every catalog size. These four interpreters name new services with 0.995–1.000 at 128 and 254 services and under both churn levels (Fig. 6 and Table 9). All 16 seen-minus-unseen contrasts stay below the 10-point bound (H5), including those of Laya, Qwen3.5-4B-JSON, and the MiniLM-Reranker. The two SemIf-Qwen3.5-4B rows are flagged because it returns no valid output there. Jev-1.13.0 detects unsupported requests less reliably, with an F1 score of 0.723–0.929 against 0.889–1.000 for the LLMs. Its median latency stays at 0.266–0.335s from 4 to 254 services. Its fee grows with the number of options, and from 64 services on it costs more per correct decision than DeepSeek-V4.1-Flash.

Option limits bound the self-hosted decision models. SemIf-Qwen3.5-4B rejects every request with more than 16

options, which removes it from the catalogs of 64 or more services and from both churn conditions. Laya rejects the 254-service catalog, and its accuracy falls from 0.981 with 4 services to 0.299 with 128. Both limits are properties of the interfaces, recorded here as observed failures.

The trained references in Table 10 show what request-time catalogs avoid. The frozen DistilBERT classifier names no new service, and its accuracy on known services is 0.653 and 0.522 at 25% and 50% churn. Retraining with 76 and 92 labelled examples, 12.8 and 10.0s of training, lifts its unseen accuracy only to 0.147 and 0.142. The MiniLM-Reranker names 0.941 and 0.933 of new services in 3 ms, but its F1 score for unsupported requests is 0.280 and 0.507.

Takeaway **Dynamic catalog.** Passing the catalog with the request lets Jev-1.13.0 and the LLMs absorb 50% churn with no loss on unseen services, whereas a classifier retrained on 92 labelled examples reaches 0.142. SemIf-Qwen3.5-4B stops at 16 options, Laya fails at 254, and Jev-1.13.0’s fee grows with the catalog. In each case the interface sets the limit.

TABLE 8. RQ3, TASK DIFFICULTY. EM AND MEDIAN DECISION LATENCY FOR EACH CONTRACT SIZE F AND CONSTRAINT DENSITY D , AND MEDIAN OUTPUT TOKENS AT MEDIUM DENSITY.

Model	EM $\uparrow$ by constraint density D			p50 (s) $\downarrow$ by constraint density D			Out. tokens
	low	medium	high	low	medium	high	$D = \text{medium}$
$F = 4$ contract fields							
Jev-1.13.0	0.923	0.957	0.967	0.279	0.278	0.268	180
SemIf-Qwen3.5-4B	0.543	0.603	0.650	0.120	0.120	0.122	0
Laya	0.253	0.207	0.157	0.055	0.055	0.056	0
DeepSeek-V4.1-Flash	0.983	0.973	0.973	0.417	0.392	0.372	25
GLM-5.3-Flash	0.897	0.987	0.997	0.916	0.645	0.645	35
Qwen3.8-Flash	0.983	0.973	0.987	1.275	1.301	1.325	30
Qwen3.5-4B-JSON (ref.)	0.530	0.677	0.677	0.766	0.764	0.767	30
$F = 6$ contract fields							
Jev-1.13.0	0.787	0.840	0.780	0.280	0.268	0.280	267
SemIf-Qwen3.5-4B	0.400	0.337	0.360	0.124	0.123	0.124	0
Laya	0.077	0.047	0.040	0.056	0.056	0.056	0
DeepSeek-V4.1-Flash	0.957	0.963	0.987	0.415	0.407	0.404	39
GLM-5.3-Flash	0.963	0.947	0.913	0.704	0.923	0.895	50
Qwen3.8-Flash	0.900	0.940	0.953	1.792	1.577	1.813	59
Qwen3.5-4B-JSON (ref.)	0.483	0.470	0.523	1.105	1.118	1.112	45
$F = 8$ contract fields							
Jev-1.13.0	0.577	0.567	0.527	0.272	0.272	0.264	364
SemIf-Qwen3.5-4B	0.133	0.100	0.070	0.133	0.134	0.135	0
Laya	0.010	0.003	0.003	0.057	0.057	0.057	0
DeepSeek-V4.1-Flash	0.900	0.930	0.930	0.460	0.455	0.483	53
GLM-5.3-Flash	0.803	0.837	0.857	0.930	0.983	1.272	71
Qwen3.8-Flash	0.823	0.790	0.633	2.092	1.859	1.965	81
Qwen3.5-4B-JSON (ref.)	0.300	0.243	0.210	1.507	1.519	1.547	63

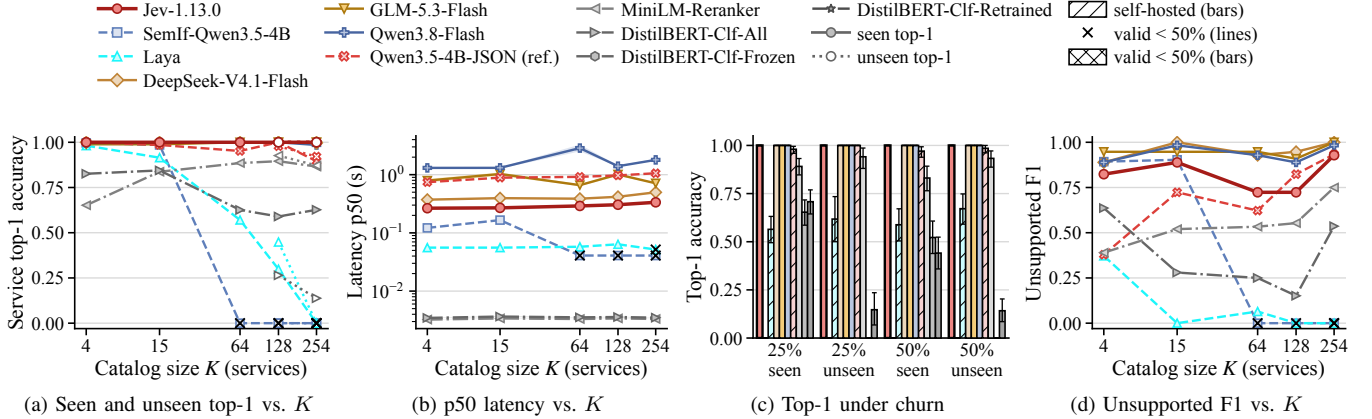


Figure 6. RQ4, dynamic service catalog. (a) Top-1 service accuracy on seen (solid) and unseen (dotted) services and (b) median decision latency as the catalog grows from 4 to 254 services; (c) top-1 accuracy when 25% or 50% of a 64-service catalog is replaced; (d) F1 score for detecting unsupported requests. Crosses mark cells with fewer than half valid outputs.

5.5. Tail Latency, Jitter, and Energy

Tail behavior follows the same order as the medians. On clean requests, Jev-1.13.0 exceeds 0.5 s in 0.7% of decisions, against 19.0% for DeepSeek-V4.1-Flash, 98.0% for GLM-5.3-Flash, and 100% for Qwen3.8-Flash (Fig. 7 and Table 11). Its interquartile range is 64 ms, against 104, 443, and 689 ms. The self-hosted decision models answer within 0.5 s every time and use 22.5 J (SemIf-Qwen3.5-4B) and 6.1 J (Laya) per decision, against 123.3 J for Qwen3.5-4B-JSON on the same accelerator (Fig. 8).

5.6. RQ5: End-to-End Service Outcome

Completion with modeled execution. Decision latency decides how much load an interpreter can admit (Fig. 9a and Table 13). With four interpretation slots, Jev-1.13.0

completes 0.907–0.953 of the requests exactly correct and on time at every offered load from 1 to 16 requests/s. DeepSeek-V4.1-Flash holds 0.907–0.983 up to 4 requests/s and falls to 0.603 at 8 and 0.093 at 16. GLM-5.3-Flash falls below 0.1 from 8 requests/s on, and Qwen3.8-Flash from 4. The completion gap to Jev-1.13.0 therefore grows with load for all three LLMs. Per doubling of λ the gap widens by 0.19, 0.25, and 0.24. The 95% confidence intervals are [0.12, 0.25], [0.22, 0.27], and [0.22, 0.25], with Holm-adjusted $p < 0.001$ (H6). At low load, DeepSeek-V4.1-Flash completes slightly more requests exactly than Jev-1.13.0 (0.983 against 0.937 at 4 requests/s), because Jev-1.13.0’s remaining semantic errors leave execution unchanged. Counting operational completion, Jev-1.13.0 reaches 0.97–1.00.

Deadlines tell the same story from the other side (Fig. 9c). At a 0.5 s deadline, Jev-1.13.0 still completes

TABLE 9. RQ4A, CATALOG SIZE. SERVICE ACCURACY ON SEEN AND UNSEEN SERVICES, UNSUPPORTED-REQUEST DETECTION, ADMISSION ERRORS, EM, VALIDITY, AND LATENCY FOR CATALOGS OF 4 TO 254 SERVICES, WITH THE TRAINED CLASSIFIER AND THE SENTENCE-EMBEDDING RERANKER AS REFERENCES.

Model	Seen top-1 $\uparrow$	Unseen top-1 $\uparrow$	Unsupported F1 $\uparrow$	False admission $\downarrow$	False blocking $\downarrow$	EM $\uparrow$	Valid $\uparrow$	p50 (s) $\downarrow$	p95 (s) $\downarrow$
<i>K = 4 services</i>									
Jev-1.13.0	1.000	–	0.824	0.300	0.000	0.920	1.000	0.266	0.351
SemIf-Qwen3.5-4B	0.996	–	0.893	0.167	0.004	0.620	1.000	0.122	0.213
Laya	0.981	–	0.372	0.733	0.019	0.207	1.000	0.056	0.151
DeepSeek-V4.1-Flash	1.000	–	0.889	0.200	0.000	0.957	1.000	0.372	0.707
GLM-5.3-Flash	0.989	–	0.947	0.100	0.000	0.947	0.987	0.790	2.944
Qwen3.8-Flash	0.996	–	0.889	0.200	0.000	0.953	0.997	1.308	2.413
Qwen3.5-4B-JSON (ref.)	1.000	–	0.378	0.767	0.000	0.467	1.000	0.744	0.879
MiniLM-Reranker	0.652	–	0.390	0.000	0.348	–	1.000	0.003	0.005
DistilBERT-Cif-All	0.826	–	0.636	0.533	0.000	–	1.000	0.003	0.006
<i>K = 15 services</i>									
Jev-1.13.0	1.000	–	0.889	0.200	0.000	0.907	1.000	0.269	0.374
SemIf-Qwen3.5-4B	0.985	–	0.903	0.067	0.015	0.643	1.000	0.166	0.246
Laya	0.915	–	0.000	1.000	0.019	0.183	1.000	0.056	0.057
DeepSeek-V4.1-Flash	1.000	–	1.000	0.000	0.000	0.987	1.000	0.395	0.608
GLM-5.3-Flash	0.989	–	0.947	0.100	0.000	0.967	0.987	1.027	4.191
Qwen3.8-Flash	0.996	–	0.983	0.033	0.000	0.950	0.997	1.310	2.259
Qwen3.5-4B-JSON (ref.)	0.985	–	0.723	0.433	0.000	0.457	1.000	0.888	1.003
MiniLM-Reranker	0.837	–	0.520	0.133	0.163	–	1.000	0.003	0.006
DistilBERT-Cif-All	0.844	–	0.280	0.767	0.048	–	1.000	0.004	0.008
<i>K = 64 services</i>									
Jev-1.13.0	1.000	–	0.723	0.433	0.000	0.867	1.000	0.290	0.378
SemIf-Qwen3.5-4B [†]	0.000	–	0.000	1.000	0.000	0.000	0.000	0.041	0.041
Laya	0.570	–	0.065	0.967	0.000	0.107	1.000	0.058	0.059
DeepSeek-V4.1-Flash	1.000	–	0.929	0.133	0.000	0.953	1.000	0.387	0.570
GLM-5.3-Flash	1.000	–	0.947	0.100	0.000	0.973	1.000	0.661	1.232
Qwen3.8-Flash	1.000	–	0.929	0.133	0.000	0.963	1.000	2.861	7.568
Qwen3.5-4B-JSON (ref.)	0.952	–	0.622	0.533	0.004	0.437	1.000	0.918	1.011
MiniLM-Reranker	0.885	–	0.533	0.467	0.052	–	1.000	0.003	0.005
DistilBERT-Cif-All	0.626	–	0.250	0.733	0.096	–	1.000	0.004	0.007
<i>K = 128 services</i>									
Jev-1.13.0	1.000	1.000	0.723	0.433	0.000	0.887	1.000	0.305	0.406
SemIf-Qwen3.5-4B [†]	0.000	0.000	0.000	1.000	0.000	0.000	0.000	0.041	0.041
Laya	0.299	0.449	0.000	1.000	0.007	0.083	1.000	0.064	0.172
DeepSeek-V4.1-Flash	1.000	1.000	0.947	0.100	0.000	0.967	1.000	0.417	0.999
GLM-5.3-Flash	1.000	1.000	0.909	0.167	0.000	0.963	1.000	1.009	2.736
Qwen3.8-Flash	1.000	1.000	0.889	0.200	0.000	0.943	1.000	1.399	2.035
Qwen3.5-4B-JSON (ref.)	1.000	0.978	0.824	0.300	0.000	0.417	1.000	0.971	1.094
MiniLM-Reranker	0.896	0.926	0.553	0.567	0.015	–	1.000	0.003	0.006
DistilBERT-Cif-All	0.590	0.265	0.152	0.833	0.115	–	1.000	0.004	0.008
<i>K = 254 services</i>									
Jev-1.13.0	1.000	1.000	0.929	0.133	0.000	0.937	1.000	0.335	0.432
SemIf-Qwen3.5-4B [†]	0.000	0.000	0.000	1.000	0.000	0.000	0.000	0.041	0.041
Laya [†]	0.000	0.000	0.000	1.000	0.000	0.000	0.000	0.052	0.052
DeepSeek-V4.1-Flash	1.000	0.995	1.000	0.000	0.000	0.970	1.000	0.498	1.008
GLM-5.3-Flash	1.000	1.000	1.000	0.000	0.000	0.980	1.000	0.711	1.876
Qwen3.8-Flash	0.985	1.000	0.983	0.033	0.000	0.963	1.000	1.795	3.791
Qwen3.5-4B-JSON (ref.)	0.896	0.921	0.933	0.067	0.007	0.493	1.000	1.059	1.179
MiniLM-Reranker	0.866	0.872	0.750	0.300	0.019	–	1.000	0.003	0.004
DistilBERT-Cif-All	0.627	0.138	0.536	0.133	0.152	–	1.000	0.003	0.004

[†]Valid output rate below 0.5; the row is printed but excluded from the best-value marking.

0.890 and DeepSeek-V4.1-Flash 0.377, while GLM-5.3-Flash and Qwen3.8-Flash complete no request. At 4s, DeepSeek-V4.1-Flash and GLM-5.3-Flash reach 0.987 and 0.990. Bursty arrivals at the center rate separate the LLMs further. DeepSeek-V4.1-Flash keeps 0.987 and Jev-1.13.0 0.950, while GLM-5.3-Flash drops from 0.807 to 0.223 and Qwen3.8-Flash stays near zero at 0.063. On the edge node, SemIf-Qwen3.5-4B and Qwen3.5-4B-JSON saturate from 2 requests/s on, since one accelerator serves the four slots in turn. Their gap therefore narrows with load (slope -0.09), and the self-hosted analogue of H6 does not hold. Laya stays fast but completes only 0.093–0.117 exactly, limited by its accuracy. Varying the number of edge nodes from 5 to 40 changes completion by less than one point when the same decision timeline is replayed. Hence the swings between the live topology cells reflect provider latency at the time of each run.

Correct completion in the real OCR service. With real recognition, the recognizer sets the ceiling (Fig. 10a–b and Table 14). Jev-1.13.0 dispatches all 180 OCR requests of every condition to a permitted node and tier, and each finishes before its deadline. Tesseract reads 92 of the 180 changing-text images exactly and 90 of the repeated ones. Jev-1.13.0’s correct completion of 0.511 and 0.500 therefore equals the recognizer’s own accuracy. Under steady arrivals with changing text, DeepSeek-V4.1-Flash completes 0.411, GLM-5.3-Flash 0.450, and Qwen3.8-Flash 0.317. Bursty arrivals cut GLM-5.3-Flash to 0.167 and Qwen3.8-Flash to 0.039, while DeepSeek-V4.1-Flash keeps 0.467 and Jev-1.13.0 0.511. On the edge node, SemIf-Qwen3.5-4B completes 0.150 and Qwen3.5-4B-JSON 0.011, both held back by the shared accelerator, and Laya completes 0.256. Because the two changing-text conditions differ only in a cache that never hits, they act as a repeated run. Jev-1.13.0

TABLE 10. RQ4B, CATALOG CHURN AT 64 SERVICES. SEEN AND UNSEEN TOP-1 ACCURACY, THEIR GAP (H5), AND THE ADAPTATION COST OF THE RETRAINED CLASSIFIER.

Model	Seen ↑	Unseen ↑	Gap [95% CI] → 0	Adapt. ex.	Adapt. (s)
25% churn					
Jev-1.13.0	1.000	1.000	0.000 [0.000, 0.000]	—	—
SemIf-Qwen3.5-4B [†]	0.000	0.000	0.000 [0.000, 0.000]	—	—
Laya	0.564	0.618	−0.053 [−0.193, 0.083]	—	—
DeepSeek-V4.1-Flash	1.000	1.000	0.000 [0.000, 0.000]	—	—
GLM-5.3-Flash	1.000	1.000	0.000 [0.000, 0.000]	—	—
Qwen3.8-Flash	1.000	1.000	0.000 [0.000, 0.000]	—	—
Qwen3.5-4B-JSON (ref.)	0.980	1.000	−0.020 [−0.041, −0.005]	—	—
MiniLM-Reranker	0.891	0.941	−0.050 [−0.117, 0.022]	—	—
DistilBERT-Clf-Frozen	0.653	0.000	0.653 [0.586, 0.717]	0	—
DistilBERT-Clf-Retrained	0.708	0.147	0.561 [0.451, 0.663]	76	12.8
50% churn					
Jev-1.13.0	1.000	1.000	0.000 [0.000, 0.000]	—	—
SemIf-Qwen3.5-4B [†]	0.000	0.000	0.000 [0.000, 0.000]	—	—
Laya	0.588	0.672	−0.083 [−0.198, 0.031]	—	—
DeepSeek-V4.1-Flash	1.000	1.000	0.000 [0.000, 0.000]	—	—
GLM-5.3-Flash	1.000	1.000	0.000 [0.000, 0.000]	—	—
Qwen3.8-Flash	1.000	1.000	0.000 [0.000, 0.000]	—	—
Qwen3.5-4B-JSON (ref.)	0.971	0.985	−0.014 [−0.053, 0.020]	—	—
MiniLM-Reranker	0.831	0.933	−0.102 [−0.179, −0.028]	—	—
DistilBERT-Clf-Frozen	0.522	0.000	0.522 [0.439, 0.607]	0	—
DistilBERT-Clf-Retrained	0.441	0.142	0.299 [0.198, 0.402]	92	10.0

[†] Valid output rate below 0.5; the row is printed but excluded from the best-value marking.

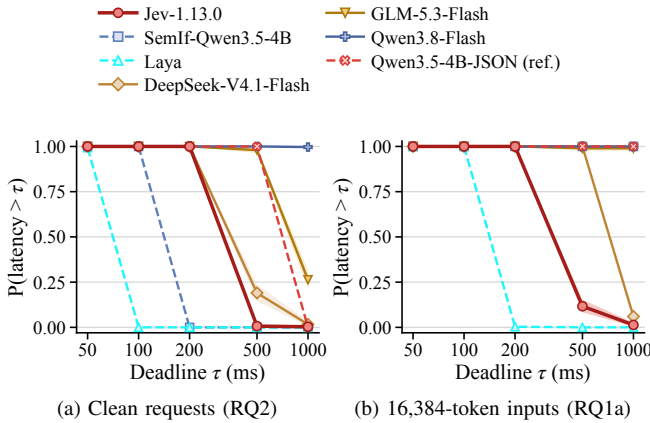


Figure 7. Decision-latency distributions as the probability that a decision exceeds a latency budget, (a) on clean requests and (b) on requests padded to 16,384 tokens.

reproduces 0.511, and DeepSeek-V4.1-Flash moves from 0.411 to 0.506 with provider latency between the runs.

No interpreter dispatched an unsupported request to OCR or sent an image to a node its locality field forbids. Jev-1.13.0 rejected all 60 counting and detection requests in every condition. The slower interpreters lost part of these rejections to the same queue expiry that costs them OCR completions. Under bursty changing text, GLM-5.3-Flash rejected 20 of the 60 in time and Qwen3.8-Flash 6.

End-to-end latency and the cache boundary. On shared successes, the requests that both Jev-1.13.0 and an LLM complete correctly, Jev-1.13.0’s 95th-percentile request latency is lower in every eligible cell of both parts (H7). The difference is 0.06–1.41 s against DeepSeek-V4.1-Flash in 16 of 16 cells and 0.50–1.52 s against GLM-5.3-Flash in 11 of 11. Because Qwen3.8-Flash shares at least 30 successes

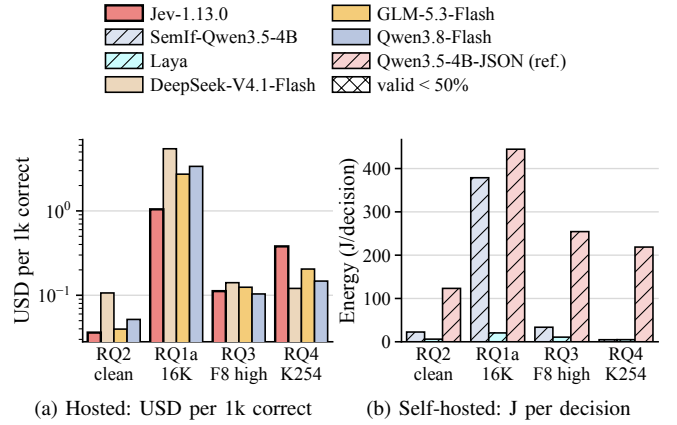


Figure 8. Efficiency per decision: (a) API fees per 1,000 correct decisions for the hosted interpreters and (b) accelerator energy per decision for the self-hosted ones, in four representative conditions.

with Jev-1.13.0 in only four cells, one short of the five that H7 requires, H7 is not evaluable for it. In those four cells its p95 is 1.29–1.48 s higher. The gap remains after image transfer, worker queueing, and recognition. Under steady changing text in Part B, the p95 differences are 0.64 s for DeepSeek-V4.1-Flash, 0.63 s for GLM-5.3-Flash, and 1.42 s for Qwen3.8-Flash (Fig. 10c). Recognition itself takes 0.07–0.08 s on average, so decision time and the admission wait it causes make up most of each request (Fig. 10d).

Caching repeated descriptions removes most of this difference (H8). With eight recurring descriptions, the cache answers 267–292 of 300 Part A requests and 229–232 of 240 Part B requests, and the median request latency of every interpreter except Laya falls to 0.06–0.09 s. The median-latency gap to Jev-1.13.0 shrinks in all nine contrasts, by 0.11–1.31 s. In both parts the completion gap closes wherever one existed. The gap shrinks by 0.22 for GLM-5.3-Flash and 0.94 for Qwen3.8-Flash in the Part A reuse block (Fig. 9d), and in Part B every interpreter except Laya reaches 0.489–0.500 with the cache on. The three completion contrasts that do not resolve start from gaps of at most 0.05 without the cache, where DeepSeek-V4.1-Flash and GLM-5.3-Flash already match Jev-1.13.0. H8 therefore holds in full for GLM-5.3-Flash and Qwen3.8-Flash in Part A and for Qwen3.8-Flash in Part B, and holds for median latency for all three LLMs. Laya stays at 0.233 with or without the cache, which replays its decisions unchanged.

Billed API cost. Fees per correct completion follow completion, because every billed call counts, including those whose request later fails. At 1 request/s, Jev-1.13.0’s API fees are 0.037 USD per 1,000 correct completions, against 0.115 for DeepSeek-V4.1-Flash, 0.043 for GLM-5.3-Flash, and 0.058 for Qwen3.8-Flash. At 16 requests/s Jev-1.13.0 stays at 0.038 USD, while the three LLMs reach 0.58, 0.65, and 2.13 USD. In Part B under steady changing text, the fees are 0.090 USD for Jev-1.13.0 against 0.309, 0.117, and 0.217 USD. These fees are higher than in Part A because every interpreter also pays for decisions whose recognition

TABLE 11. COMMUNICATION-LEVEL METRICS IN FOUR REPRESENTATIVE CONDITIONS: TAIL LATENCY, JITTER, PROBABILITY OF EXCEEDING A LATENCY BUDGET, AVAILABILITY, GOODPUT, SLA VIOLATIONS, PAYLOAD BYTES, API FEES, AND ENERGY PER DECISION.

Model	p95 (s) ↓	p99 (s) ↓	Jitter IQR (s) ↓	P(>0.2 s) ↓	P(>1 s) ↓	Avail. ↑	Goodput (corr./s) ↑	SLA violation ↓	Request (B) ↓	Response (B) ↓	USD per 1k correct ↓	Energy (J/dec.) ↓
RQ2, clean												
Jev-1.13.0	0.370	0.438	0.064	1.000	0.003	1.000	3.33	0.003	186.5	728.5	0.03612	–
SemIf-Qwen3.5-4B	0.123	0.123	0.001	0.000	0.000	1.000	5.27	0.080	186.5	1031	–	22.5
Laya	0.056	0.056	0.000	0.000	0.000	1.000	2.01	0.143	186.5	849	–	6.1
DeepSeek-V4.1-Flash	0.692	1.280	0.104	1.000	0.017	1.000	2.19	0.003	186.5	927	0.1065	–
GLM-5.3-Flash	2.940	3.982	0.443	1.000	0.263	1.000	0.93	0.003	186.5	1115	0.03971	–
Qwen3.8-Flash	3.450	5.099	0.689	1.000	0.997	0.997	0.49	0.003	186.5	925	0.05172	–
Qwen3.5-4B-JSON (ref.)	0.811	0.833	0.043	1.000	0.000	1.000	0.63	0.053	186.5	511	–	123.3
RQ1a, 16,384-token input												
Jev-1.13.0	0.523	1.198	0.055	1.000	0.013	1.000	1.89	0.013	41749	740	1.044	–
SemIf-Qwen3.5-4B	1.238	1.288	0.016	1.000	1.000	1.000	0.41	0.340	41749	1033	–	378.8
Laya	0.140	0.142	0.002	0.003	0.000	1.000	0.12	0.500	41749	852	–	20.4
DeepSeek-V4.1-Flash	1.023	1.183	0.170	1.000	0.060	1.000	1.40	0.003	41749	937	5.455	–
GLM-5.3-Flash	3.939	12.754	0.829	1.000	0.990	0.980	0.44	0.050	41749	937	2.727	–
Qwen3.8-Flash	4.274	5.400	0.670	1.000	1.000	1.000	0.31	0.000	41749	945	3.373	–
Qwen3.5-4B-JSON (ref.)	1.619	2.384	0.024	1.000	1.000	1.000	0.11	0.127	41749	514	–	444.7
RQ3, F = 8, high density												
Jev-1.13.0	0.350	0.401	0.047	1.000	0.000	1.000	1.91	0.057	402.5	1313	0.1119	–
SemIf-Qwen3.5-4B	0.221	0.267	0.040	0.080	0.000	1.000	0.47	0.223	402.5	1935	–	33.6
Laya	0.193	0.224	0.021	0.040	0.000	1.000	0.04	0.533	402.5	1544	–	10.7
DeepSeek-V4.1-Flash	0.953	7.556	0.091	1.000	0.050	1.000	1.45	0.010	402.5	1059	0.1410	–
GLM-5.3-Flash	2.462	4.848	0.449	1.000	0.843	1.000	0.60	0.100	402.5	1405.5	0.1245	–
Qwen3.8-Flash	5.592	7.078	0.517	1.000	1.000	0.960	0.27	0.050	402.5	1079.5	0.1037	–
Qwen3.5-4B-JSON (ref.)	1.657	1.951	0.074	1.000	1.000	1.000	0.13	0.113	402.5	636.5	–	254.4
RQ4, K = 254 services												
Jev-1.13.0	0.432	0.502	0.046	1.000	0.003	1.000	2.65	0.013	245.5	8161	0.3798	–
SemIf-Qwen3.5-4B†	0.041	0.041	0.000	0.000	0.000	0.000	0.00	1.000	245.5	128	–	4.8
Laya†	0.052	0.053	0.000	0.000	0.000	0.000	0.00	1.000	245.5	119	–	4.9
DeepSeek-V4.1-Flash	1.008	1.574	0.090	1.000	0.053	1.000	1.78	0.003	245.5	965	0.1206	–
GLM-5.3-Flash	1.876	2.699	0.202	1.000	0.163	1.000	1.13	0.017	245.5	1153	0.2046	–
Qwen3.8-Flash	3.791	5.274	0.656	1.000	1.000	1.000	0.47	0.007	245.5	957	0.1473	–
Qwen3.5-4B-JSON (ref.)	1.179	1.671	0.075	1.000	0.867	1.000	0.45	0.053	245.5	529	–	218.7

†Valid output rate below 0.5; the row is printed but excluded from the best-value marking.

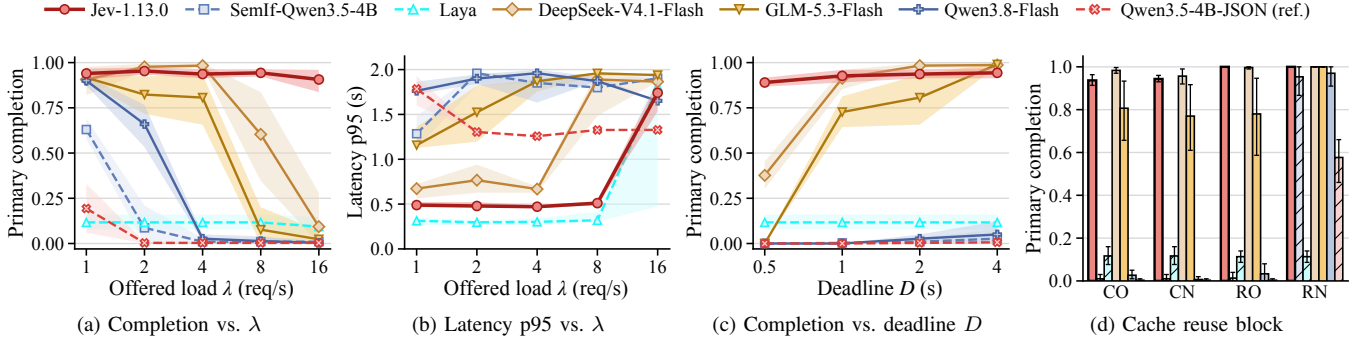


Figure 9. RQ5, modeled execution (Part A). (a) Primary completion and (b) 95th percentile of full request latency T on completions as offered load λ increases from 1 to 16 requests/s. (c) Primary completion across request deadline D from 0.5 to 4 s ($D = 2$ s is the $\lambda = 4$ cell). Bands are 95% bootstrap confidence intervals. (d) Primary completion under cache reuse conditions: CO changing text with cache off, CN changing text with cache on, RO repeated text with cache off, RN repeated text with cache on.

fails. With repeated descriptions cached, all four fall to 0.001–0.009 USD. Of the 19,410 hosted calls, 24 reported no usage and are left out of the fees.

Takeaway End-to-end service. Decision time decides how much load an admission path can take. Jev-1.13.0 keeps 0.91–0.95 exact, on-time completion from 1 to 16 requests/s, where the LLMs fall below 0.1. In the real OCR service it reaches the recognizer’s own accuracy under every arrival pattern. Because caching repeated descriptions gives the interpreters nearly the same latency, Jev-1.13.0’s advantage lies in requests that need a fresh decision.

6. Discussion

6.1. Implications for Service Orchestration

The experiments show a practical use for Jev in edge-service admission, where it reduces the time and API fees spent interpreting requests on bounded contracts. The shared intent contract lets Jev replace the generative interpreter within the same validation and scheduling pipeline. The real OCR results show that the latency benefit remains visible after image transfer, worker queueing, and recognition. On requests that both complete, Jev-1.13.0’s 95th-percentile

TABLE 12. PRE-REGISTERED CONFIRMATORY CONTRASTS H1–H5 (THE PAIRED H3 CONTRASTS ARE IN TABLE 7). A CONTRAST IS RESOLVED WHEN ITS 95% CONFIDENCE INTERVAL EXCLUDES THE NULL VALUE AND ITS HOLM-ADJUSTED p IS BELOW 0.05.

Contrast	Level	Estimate [95% CI]	Holm p	Verdict
H1(a): paired p50 latency difference (s)				
DeepSeek-V4.1-Flash — Jev-1.13.0	Base length	0.109 [0.102, 0.117]	3.8e-46	resolved
DeepSeek-V4.1-Flash — Jev-1.13.0	Padded to 512 tokens	0.217 [0.204, 0.231]	5.8e-48	resolved
DeepSeek-V4.1-Flash — Jev-1.13.0	Padded to 2,048 tokens	0.269 [0.257, 0.283]	6.7e-49	resolved
DeepSeek-V4.1-Flash — Jev-1.13.0	Padded to 8,192 tokens	0.165 [0.151, 0.183]	6.8e-42	resolved
DeepSeek-V4.1-Flash — Jev-1.13.0	Padded to 16,384 tokens	0.218 [0.204, 0.236]	1.2e-45	resolved
GLM-5.3-Flash — Jev-1.13.0	Base length	0.370 [0.347, 0.396]	4.9e-49	resolved
GLM-5.3-Flash — Jev-1.13.0	Padded to 512 tokens	0.447 [0.425, 0.470]	1.2e-49	resolved
GLM-5.3-Flash — Jev-1.13.0	Padded to 2,048 tokens	0.400 [0.370, 0.427]	1.2e-49	resolved
GLM-5.3-Flash — Jev-1.13.0	Padded to 8,192 tokens	0.751 [0.722, 0.811]	6.2e-49	resolved
GLM-5.3-Flash — Jev-1.13.0	Padded to 16,384 tokens	1.156 [1.072, 1.218]	6.7e-47	resolved
Qwen3.8-Flash — Jev-1.13.0	Base length	1.472 [1.379, 1.616]	1.2e-49	resolved
Qwen3.8-Flash — Jev-1.13.0	Padded to 512 tokens	1.358 [1.294, 1.459]	1.2e-49	resolved
Qwen3.8-Flash — Jev-1.13.0	Padded to 2,048 tokens	1.314 [1.274, 1.349]	1.2e-49	resolved
Qwen3.8-Flash — Jev-1.13.0	Padded to 8,192 tokens	1.994 [1.946, 2.066]	1.2e-49	resolved
Qwen3.8-Flash — Jev-1.13.0	Padded to 16,384 tokens	2.497 [2.416, 2.553]	5.8e-48	resolved
Qwen3.5-4B-JSON — SemIf-Qwen3.5-4B	Base length	0.648 [0.635, 0.649]	1.2e-49	resolved
Qwen3.5-4B-JSON — SemIf-Qwen3.5-4B	Padded to 512 tokens	0.637 [0.633, 0.646]	1.2e-49	resolved
Qwen3.5-4B-JSON — SemIf-Qwen3.5-4B	Padded to 2,048 tokens	0.645 [0.643, 0.646]	1.2e-49	resolved
Qwen3.5-4B-JSON — SemIf-Qwen3.5-4B	Padded to 8,192 tokens	0.534 [0.532, 0.537]	1.2e-49	resolved
Qwen3.5-4B-JSON — SemIf-Qwen3.5-4B	Padded to 16,384 tokens	0.393 [0.391, 0.395]	1.2e-49	resolved
H1(b): ratio of latency growth factors				
$G(\text{Jev-1.13.0; 16K}) / G(\text{DeepSeek-V4.1-Flash; 16K})$	16,384 tokens vs base	0.904 [0.865, 0.940]	$\leq 4.0e-4$	resolved
$G(\text{Jev-1.13.0; 16K}) / G(\text{GLM-5.3-Flash; 16K})$	16,384 tokens vs base	0.622 [0.585, 0.663]	$\leq 4.0e-4$	resolved
$G(\text{Jev-1.13.0; 16K}) / G(\text{Qwen3.8-Flash; 16K})$	16,384 tokens vs base	0.929 [0.866, 0.986]	0.018	resolved
$G(\text{SemIf-Qwen3.5-4B; 16K}) / G(\text{Qwen3.5-4B-JSON; 16K})$	16,384 tokens vs base	4.675 [4.665, 4.701]	$\leq 4.0e-4$	not resolved
H2: ratio of latency growth factors				
$R(\text{Jev-1.13.0; 8}) / R(\text{DeepSeek-V4.1-Flash; 8})$	$k = 8$ vs $k = 1$	0.494 [0.468, 0.519]	$\leq 4.0e-4$	resolved
$R(\text{Jev-1.13.0; 8}) / R(\text{GLM-5.3-Flash; 8})$	$k = 8$ vs $k = 1$	0.399 [0.379, 0.427]	$\leq 4.0e-4$	resolved
$R(\text{Jev-1.13.0; 8}) / R(\text{Qwen3.8-Flash; 8})$	$k = 8$ vs $k = 1$	0.367 [0.350, 0.387]	$\leq 4.0e-4$	resolved
$R(\text{SemIf-Qwen3.5-4B; 8}) / R(\text{Qwen3.5-4B-JSON; 8})$	$k = 8$ vs $k = 1$	0.528 [0.515, 0.575]	$\leq 4.0e-4$	resolved
H3: Cochran's Q on unsafe decisions (statistic)				
Cochran's Q on unsafe across six	Clean	188.3	7.2e-38	resolved
Cochran's Q on unsafe across six	Code-switched	171.9	1.7e-34	resolved
Cochran's Q on unsafe across six	Colloquial	98.0	4.2e-19	resolved
Cochran's Q on unsafe across six	Default-baiting	82.7	4.6e-16	resolved
Cochran's Q on unsafe across six	Key-value	50.0	1.4e-9	resolved
Cochran's Q on unsafe across six	Negation-heavy	120.3	1.1e-23	resolved
Cochran's Q on unsafe across six	Noisy	171.0	2.2e-34	resolved
Cochran's Q on unsafe across six	Self-revising	172.2	1.7e-34	resolved
H4: paired output-token difference (tokens)				
DeepSeek-V4.1-Flash output tokens F8 — F4	$F = 8$ vs $F = 4$, all D	28.0 [27.0, 28.0]	$\leq 3.0e-4$	resolved
GLM-5.3-Flash output tokens F8 — F4	$F = 8$ vs $F = 4$, all D	36.0 [35.0, 37.0]	$\leq 3.0e-4$	resolved
Qwen3.8-Flash output tokens F8 — F4	$F = 8$ vs $F = 4$, all D	43.0 [42.0, 44.5]	$\leq 3.0e-4$	resolved
H4: ratio of latency growth factors				
$G(\text{Jev-1.13.0; F8/F4}) / G(\text{DeepSeek-V4.1-Flash; F8/F4})$	$F = 8$ vs $F = 4$, all D	0.828 [0.807, 0.844]	$\leq 4.0e-4$	resolved
$G(\text{Jev-1.13.0; F8/F4}) / G(\text{GLM-5.3-Flash; F8/F4})$	$F = 8$ vs $F = 4$, all D	0.638 [0.620, 0.652]	$\leq 4.0e-4$	resolved
$G(\text{Jev-1.13.0; F8/F4}) / G(\text{Qwen3.8-Flash; F8/F4})$	$F = 8$ vs $F = 4$, all D	0.650 [0.633, 0.670]	$\leq 4.0e-4$	resolved
$G(\text{SemIf-Qwen3.5-4B; F8/F4}) / G(\text{Qwen3.5-4B-JSON; F8/F4})$	$F = 8$ vs $F = 4$, all D	0.561 [0.559, 0.562]	$\leq 4.0e-4$	resolved
H5: seen — unseen top-1 gap (proportion)				
Jev-1.13.0 seen — unseen gap	25% churn	0.000 [0.000, 0.000]	≤ 0.0016	resolved
DeepSeek-V4.1-Flash seen — unseen gap	25% churn	0.000 [0.000, 0.000]	≤ 0.0016	resolved
GLM-5.3-Flash seen — unseen gap	25% churn	0.000 [0.000, 0.000]	≤ 0.0016	resolved
Qwen3.8-Flash seen — unseen gap	25% churn	0.000 [0.000, 0.000]	≤ 0.0016	resolved
SemIf-Qwen3.5-4B seen — unseen gap [†]	25% churn	0.000 [0.000, 0.000]	≤ 0.0016	resolved
Laya seen — unseen gap	25% churn	−0.053 [−0.193, 0.083]	0.027	resolved
Qwen3.5-4B-JSON seen — unseen gap	25% churn	−0.020 [−0.041, −0.005]	≤ 0.0016	resolved
MiniLM-Reranker seen — unseen gap	25% churn	−0.050 [−0.117, 0.022]	≤ 0.0016	resolved
Jev-1.13.0 seen — unseen gap	50% churn	0.000 [0.000, 0.000]	≤ 0.0016	resolved
DeepSeek-V4.1-Flash seen — unseen gap	50% churn	0.000 [0.000, 0.000]	≤ 0.0016	resolved
GLM-5.3-Flash seen — unseen gap	50% churn	0.000 [0.000, 0.000]	≤ 0.0016	resolved
Qwen3.8-Flash seen — unseen gap	50% churn	0.000 [0.000, 0.000]	≤ 0.0016	resolved
SemIf-Qwen3.5-4B seen — unseen gap [†]	50% churn	0.000 [0.000, 0.000]	≤ 0.0016	resolved
Laya seen — unseen gap	50% churn	−0.083 [−0.198, 0.031]	0.0016	resolved
Qwen3.5-4B-JSON seen — unseen gap	50% churn	−0.014 [−0.053, 0.020]	≤ 0.0016	resolved
MiniLM-Reranker seen — unseen gap	50% churn	−0.102 [−0.179, −0.028]	≤ 0.0016	resolved

[†]Valid output rate below 0.5; the row is printed but excluded from the best-value marking.

request latency stays 0.63–1.42 s below the LLMs' under steady arrivals with changing text.

The interpretation layer also marks where the substitution stops paying. Jev-1.13.0's decision time barely moves with input length, bundling, contract size, or catalog size. Hence its advantage grows where generative interpreters slow down. Its accuracy does move with contract size. With

four fields it trails the strongest LLM by a few points, and with eight by more than 30. A service designer can therefore place Jev-1.13.0 on short contracts and latency-bound paths, and route wide contracts to a generative interpreter. Request-time catalogs favor both families over a trained classifier, since neither needs labelled examples for a new service.

This integration gives service designers two ways to

TABLE 13. RQ5, MODELED EXECUTION (PART A). PRIMARY COMPLETION RATE AND 95TH PERCENTILE REQUEST LATENCY T ON COMPLETIONS ACROSS OFFERED LOAD, DEADLINE, TOPOLOGY, CACHE REUSE, AND BURSTINESS SWEEPS. THE $\lambda = 4$ CELL SERVES AS THE REFERENCE FOR $D = 2$ s, $N = 5$, AND CHANGING TEXT WITH CACHE OFF. MODEL CODES: JEV JEV-1.13.0, SIF SEMIF-QWEN3.5-4B, LAY LAYA, DSK DEEPSEEK-V4.1-FLASH, GLM GLM-5.3-FLASH, Q38 QWEN3.8-FLASH, QJS QWEN3.5-4B-JSON (REF.). BEST VALUE PER ROW WITHIN EACH METRIC BLOCK IN BOLD (QJS REFERENCE EXCLUDED). 95% CONFIDENCE INTERVALS ARE DRAWN IN FIG. 9.

Cell	Primary completion $\uparrow$							p95 T (s) $\downarrow$						
	JEV	SIF	LAY	DSK	GLM	Q38	QJS	JEV	SIF	LAY	DSK	GLM	Q38	QJS
Offered load sweep ($\lambda \in \{1, 2, 4, 8, 16\}$ req/s)														
$\lambda=1$	0.940	0.630	0.117	0.907	0.917	0.900	0.193	0.49	1.28	0.31	0.67	1.16	1.76	1.78
$\lambda=2$	0.953	0.087	0.117	0.977	0.823	0.660	0.003	0.48	1.96	0.30	0.77	1.52	1.90	1.30
$\lambda=4$	0.937	0.010	0.117	0.983	0.807	0.027	0.003	0.47	1.85	0.30	0.67	1.87	1.96	1.26
$\lambda=8$	0.943	0.010	0.117	0.603	0.077	0.013	0.003	0.51	1.80	0.32	1.89	1.96	1.87	1.33
$\lambda=16$	0.907	0.010	0.093	0.093	0.023	0.003	0.003	1.74	1.90	1.82	1.87	1.94	1.65	1.33
Deadline sweep ($D \in \{0.5, 1, 4\}$ s; $D = 2$ s in load sweep)														
$D=0.5$ s	0.890	0.000	0.117	0.377	0.000	0.000	0.000	0.46	–	0.29	0.49	–	–	–
$D=1$ s	0.927	0.003	0.117	0.913	0.727	0.000	0.000	0.47	0.89	0.29	0.83	0.97	–	–
$D=4$ s	0.943	0.027	0.117	0.987	0.990	0.050	0.007	0.48	3.83	0.30	1.01	1.45	3.95	2.89
Topology sweep ($N \in \{10, 20, 40\}$ nodes; $N = 5$ in load sweep)														
$N=10$	0.940	0.010	0.117	0.993	0.023	0.013	0.003	0.48	1.82	0.35	1.48	1.92	1.92	1.28
$N=20$	0.943	0.010	0.117	0.853	0.123	0.010	0.003	0.50	1.84	0.34	1.85	1.99	1.71	1.26
$N=40$	0.947	0.010	0.117	0.990	0.023	0.007	0.003	0.51	1.84	0.32	1.39	1.91	1.64	1.38
Cache reuse sweep ($\lambda = 4$, $D = 2$ s; CO in load sweep)														
changing, on	0.943	0.010	0.117	0.957	0.770	0.007	0.003	0.47	1.65	0.30	0.80	1.79	1.93	1.32
repeated, off	1.000	0.013	0.113	0.997	0.780	0.033	0.003	0.48	1.89	0.30	0.70	1.83	1.86	1.51
repeated, on	1.000	0.953	0.113	1.000	1.000	0.970	0.577	0.20	0.19	0.20	0.20	0.20	0.19	0.19
Burstiness sweep ($\lambda = 4$, $D = 2$ s, Poisson bursts)														
bursty	0.950	0.083	0.117	0.987	0.223	0.063	0.053	0.51	1.49	0.33	0.97	1.93	1.93	1.77

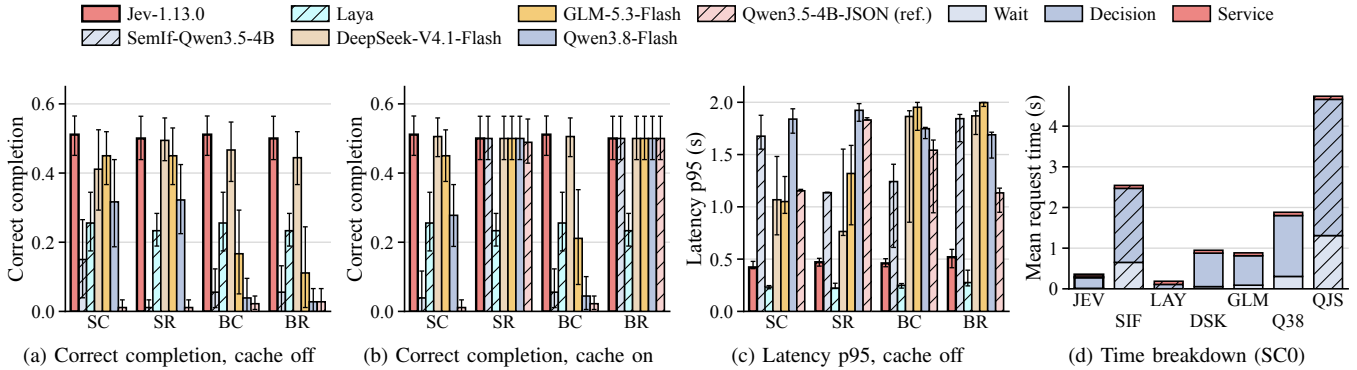


Figure 10. RQ5, real OCR service (Part B). (a) Correct-completion rate with cache off (whiskers are 95% confidence intervals) and (b) cache on across operational conditions: S steady, B bursty, C changing text, R repeated text. (c) 95th percentile request latency T on completions with cache off. (d) Stacked mean time decomposition into admission wait, decision, and service execution for SC0. Model codes: JEV JEV-1.13.0, SIF SEMIF-QWEN3.5-4B, LAY LAYA, DSK DEEPSEEK-V4.1-FLASH, GLM GLM-5.3-FLASH, Q38 QWEN3.8-FLASH, QJS QWEN3.5-4B-JSON (REF.). BEST VALUE PER ROW WITHIN EACH METRIC BLOCK IN BOLD (QJS REFERENCE EXCLUDED). 95% CONFIDENCE INTERVALS ARE DRAWN IN FIG. 10.

TABLE 14. RQ5, REAL OCR SERVICE (PART B). CORRECT COMPLETION RATE AND 95TH PERCENTILE REQUEST LATENCY T ON COMPLETIONS ACROSS OPERATIONAL CONDITIONS UNDER STEADY AND BURSTY ARRIVALS. MODEL CODES: JEV JEV-1.13.0, SIF SEMIF-QWEN3.5-4B, LAY LAYA, DSK DEEPSEEK-V4.1-FLASH, GLM GLM-5.3-FLASH, Q38 QWEN3.8-FLASH, QJS QWEN3.5-4B-JSON (REF.). BEST VALUE PER ROW WITHIN EACH METRIC BLOCK IN BOLD (QJS REFERENCE EXCLUDED). 95% CONFIDENCE INTERVALS ARE DRAWN IN FIG. 10.

Condition	Correct completion $\uparrow$							p95 T (s) $\downarrow$						
	JEV	SIF	LAY	DSK	GLM	Q38	QJS	JEV	SIF	LAY	DSK	GLM	Q38	QJS
Steady arrivals														
changing, off	0.511	0.150	0.256	0.411	0.450	0.317	0.011	0.42	1.68	0.23	1.07	1.05	1.84	1.15
changing, on	0.511	0.039	0.256	0.506	0.450	0.278	0.011	0.48	1.72	0.22	0.69	1.57	1.95	1.20
repeated, off	0.500	0.011	0.233	0.494	0.450	0.322	0.011	0.47	1.14	0.22	0.76	1.32	1.92	1.83
repeated, on	0.500	0.500	0.233	0.500	0.500	0.500	0.489	0.13	0.13	0.13	0.12	0.13	0.13	0.13
Bursty arrivals														
changing, off	0.511	0.056	0.256	0.467	0.167	0.039	0.022	0.46	1.24	0.25	1.86	1.95	1.75	1.54
changing, on	0.511	0.056	0.256	0.506	0.211	0.044	0.022	0.45	1.26	0.26	1.36	1.96	1.98	1.54
repeated, off	0.500	0.056	0.233	0.444	0.111	0.028	0.028	0.52	1.84	0.28	1.87	2.00	1.69	1.13
repeated, on	0.500	0.500	0.233	0.500	0.500	0.500	0.500	0.14	0.16	0.10	0.14	0.15	0.13	0.15

reduce interpretation overhead. A faster decision backend shortens fresh calls, while caching reuses decisions for repeated descriptions. With eight recurring descriptions, the cache answers nearly every request. Every interpreter except Laya, including the self-hosted Qwen3.5-4B-JSON, then

completes 0.489–0.500, the recognizer’s accuracy. Choosing between backends therefore depends on the share of requests that need a new decision and the time available for service execution. In the admission path studied here, shorter decision waiting leaves more deadline slack and releases

interpretation capacity sooner.

6.2. Evaluating the Complete Service Path

The shared scheduler connects interpreter performance to delivered service. It reads current worker state after interpretation and checks placement, tier, priority, and predicted completion time before dispatch. Evaluating intent fidelity, execution compliance, and actual output separately makes it possible to trace how an interpretation affects the final result. For example, the normal-versus-unspecified urgency difference maps to the same execution priority, while the returned OCR text determines whether recognition succeeded. This evaluation supports a service-level choice of interpreter using completion, response time, and fees together.

6.3. Scope and Next Steps

Because EdgeIntent v1 is generated and verified by language models, its wording may favor phrasing that such models parse easily. Programmatic padding and noise conditions and the blind agreement filter limit this effect. Each case is run once per interpreter, and the service layer uses one arrival trace per cell. The real service has three workers and one service family. Hosted timings include provider and network paths, with sequential runs exposed to temporal variation. They compare deployed services rather than isolate internal model inference. API fees cover interpretation calls, whereas local compute and communication costs are outside this measure. Section 4.7 records the configuration.

The next step is to evaluate independently authored requests across days, services, and network conditions, using deadlines drawn from application requirements. Optimized local serving, a bounded extractor, and decision models with larger option limits than SemIf-Qwen3.5-4B and Laya would extend the available deployment choices.

7. Conclusion

We integrated Jev-1.13.0 into an edge-service admission pipeline and compared it with two self-hosted decision models and three hosted LLMs on EdgeIntent v1 and on a live admission path. Jev-1.13.0's median decision latency is 22.7–64.5% below that of DeepSeek-V4.1-Flash and barely moves with input length, bundling, contract size, or catalog size. On four-field contracts, Jev-1.13.0 also cuts API fees per correct decision by 59.7–80.9% and trails DeepSeek-V4.1-Flash by a few exact-match points. Wide contracts bound the substitution, because the accuracy gap exceeds 30 points at eight fields. Request-time catalogs let Jev-1.13.0 name unseen services as accurately as known ones. On the live admission path, Jev-1.13.0 keeps 0.91–0.95 of requests exact and on time at up to 16 requests/s, whereas the LLMs fall below 0.1. Caching repeated descriptions gives the interpreters nearly the same latency, which confines Jev-1.13.0's advantage to requests that need fresh interpretation. A service designer can therefore use Jev-1.13.0 for latency-bound admission on short contracts, pair it with caching for

recurring requests, and route wide contracts to a generative interpreter.

References

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.
- [3] J. Santos, T. Wauters, B. Volckaert, and F. De Turck, "Towards low-latency service delivery in a continuum of virtual resources: State-of-the-art and research directions," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2557–2589, 2021.
- [4] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, "Intent-based networking - concepts and definitions," RFC Editor, RFC 9315, Oct. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9315>
- [5] A. S. Jacobs, R. J. Pfitscher, R. H. Ribeiro, R. A. Ferreira, L. Z. Granville, W. Willinger, and S. G. Rao, "Hey, Lumi! using natural language for intent-based network management," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, Jul. 2021, pp. 625–639. [Online]. Available: <https://www.usenix.org/conference/atc21/presentation/jacobs>
- [6] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs facilitate network configuration?" *Proceedings of the ACM on Networking*, vol. 2, no. CoNEXT2, pp. 1–25, Jun. 2024.
- [7] A. Angi, A. Sacco, and G. Marchetto, "LLNet: An intent-driven approach to instructing softwarized network devices using a small language model," *IEEE Transactions on Network and Service Management*, vol. 22, pp. 3403–3418, Aug. 2025.
- [8] Y. Miyaoka, M. Inoue, K. Urata, and S. Harada, "Chat-driven optimal management for virtual network services," arXiv preprint arXiv:2512.24614, 2025. [Online]. Available: <https://arxiv.org/abs/2512.24614>
- [9] L. Nisiotis and A. Hadjilias, "From prompt to service: An SLM-based agent orchestration gateway for AI-driven virtual worlds," arXiv preprint arXiv:2606.03557, Sep. 2026. [Online]. Available: <https://arxiv.org/abs/2606.03557>
- [10] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y. C. Lin, and P. Molchanov, "Small language models are the future of agentic AI," arXiv preprint arXiv:2506.02153, 2025. [Online]. Available: <https://arxiv.org/abs/2506.02153>
- [11] U. Zaratiana, G. Pasternak, O. Boyd, G. Hurn-Maloney, and A. Lewis, "GLiNER2: Schema-driven multi-task learning for structured information extraction," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Suzhou, China: Association for Computational Linguistics, 2025, pp. 130–140.
- [12] D. A. Friedman, "Jev in practice: A composable Python toolkit for TypeSafe's System One decision model," Zenodo, Sep. 2026. [Online]. Available: <https://zenodo.org/records/22817425>
- [13] T. Lee, "SemIf (formerly OpenJev)," <https://github.com/TheoLeeCJ/SemIf-OpenJev>, 2026, commit 23cf1f39.
- [14] Convai Innovations, "Laya Typed-Decisions," <https://huggingface.co/convaiinnovations/laya-typed-decisions>, 2026, revision bc76315b.
- [15] D. M. Manias, A. Chouman, and A. Shami, "Towards intent-based network management: Large language models for intent extraction in 5G core networks," in *2024 20th International Conference on the Design of Reliable Communication Networks (DRCN)*. Montreal, QC, Canada: IEEE, May 2024, pp. 1–6.

- [16] O. G. Lira, O. M. Caicedo, and N. L. S. Da Fonseca, "Network self-configuration based on fine-tuned small language models," arXiv preprint arXiv:2512.02861, 2025. [Online]. Available: <https://arxiv.org/abs/2512.02861>
- [17] D. Brodimas, A. Birbas, D. Kaposos, and S. Denazis, "Intent-based infrastructure and service orchestration using agentic-AI," *IEEE Open Journal of the Communications Society*, vol. 6, pp. 7150–7168, 2025.
- [18] E. Li and H. Du, "JAUNT: Joint alignment of user intent and network state for QoE-centric LLM tool routing," arXiv preprint arXiv:2510.18550, 2025. [Online]. Available: <https://arxiv.org/abs/2510.18550>
- [19] K. Islam and R. N. Calheiros, "Intent Engine: Natural-language intent translation for intent-driven orchestration in the compute continuum," *Journal of Systems Architecture*, vol. 179, p. 103938, Oct. 2026.
- [20] J. Martins, L. Mokrushin, M. Orlic, and A. K. A., "Intent-driven 6G service orchestration: Grounded translation, validation, and decomposition," arXiv preprint arXiv:2606.28348, Jun. 2026. [Online]. Available: <https://arxiv.org/abs/2606.28348>
- [21] A. Mekrache, A. Ksentini, and C. Verikoukis, "DMO-GPT: An intent-driven framework for distributed 6G management and orchestration," *IEEE Communications Magazine*, vol. 64, no. 1, pp. 48–54, Jan. 2026.
- [22] J. Parra-Ullauri, T. A. Khan, D. McHugh, S. Kapoor, A. Duke, A. Hey, and A. Corston-Petrie, "Role-based agentic AI for intent-driven network and service orchestration," arXiv preprint arXiv:2606.20580, 2026. [Online]. Available: <https://arxiv.org/abs/2606.20580>
- [23] D. Seo and K. Kim, "An integrated pipeline for intent-based zero-touch networks: From intent translation to minimal-modification re-configuration," *Applied Sciences*, vol. 16, p. 5811, Jun. 2026.
- [24] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting large language models for networking," in *Proceedings of the ACM SIGCOMM 2024 Conference*. Sydney NSW Australia: ACM, Aug. 2024, pp. 661–678.
- [25] S. Geng, H. Cooper, M. Moskal, S. Jenkins, J. Berman, N. Ranchin, R. West, E. Horvitz, and H. Nori, "JSONSchemaBench: A rigorous benchmark of structured outputs for language models," arXiv preprint arXiv:2501.10868, 2025. [Online]. Available: <https://arxiv.org/abs/2501.10868>
- [26] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A low-latency online prediction serving system," arXiv preprint arXiv:1612.03079, 2016. [Online]. Available: <https://arxiv.org/abs/1612.03079>
- [27] D. Crankshaw, G.-E. Sela, X. Mo, C. Zumar, I. Stoica, J. Gonzalez, and A. Tumanov, "InferLine: Latency-aware provisioning and scaling for prediction serving pipelines," in *Proceedings of the 11th ACM Symposium on Cloud Computing*. Virtual Event USA: ACM, Oct. 2020, pp. 477–491.
- [28] F. Bang, "GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings," in *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*. Singapore, Singapore: Association for Computational Linguistics, 2023, pp. 212–218.
- [29] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv preprint arXiv:2305.05176, 2023. [Online]. Available: <https://arxiv.org/abs/2305.05176>
- [30] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," arXiv preprint arXiv:2406.18665, 2024. [Online]. Available: <https://arxiv.org/abs/2406.18665>
- [31] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, pp. 3980–3990.
- [32] L. Tunstall, N. Reimers, U. E. S. Jo, L. Bates, D. Korat, M. Wasserblat, and O. Pereg, "Efficient few-shot learning without prompts," arXiv preprint arXiv:2209.11055, 2022. [Online]. Available: <https://arxiv.org/abs/2209.11055>
- [33] U. Zaratiana, N. Tomeh, P. Holat, and T. Charnois, "GLiNER: Generalist model for named entity recognition using bidirectional transformer," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City, Mexico: Association for Computational Linguistics, 2024, pp. 5364–5376.
- [34] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019.
- [35] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," 2020.
- [36] Qwen Team, "Qwen3.5-4B," <https://huggingface.co/Qwen/Qwen3.5-4B>, 2026, revision 851bf6e8.
- [37] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*. Koblenz Germany: ACM, Oct. 2023, pp. 611–626.
- [38] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," arXiv preprint arXiv:2312.07104, 2023. [Online]. Available: <https://arxiv.org/abs/2312.07104>
- [39] L. Chen, M. Zaharia, and J. Zou, "FrugalML: How to use ML prediction APIs more accurately and cheaply," arXiv preprint arXiv:2006.07512, 2020. [Online]. Available: <https://arxiv.org/abs/2006.07512>
- [40] R. Smith, "An overview of the Tesseract OCR engine," in *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007) Vol 2*. Curitiba, Parana, Brazil: IEEE, Sep. 2007, pp. 629–633.
- [41] A. Mishra, K. Alahari, and C. Jawahar, "Scene text recognition using higher order language priors," in *Proceedings of the British Machine Vision Conference*. Surrey: British Machine Vision Association, 2012, pp. 127.1–127.11.